From RNNs to Transformers

Self-Attention and Positional Encoding

Professor Anis Koubaa

MAI554: Deep Learning for NLP
Alfaisal University
akoubaa@alfaisal.edu

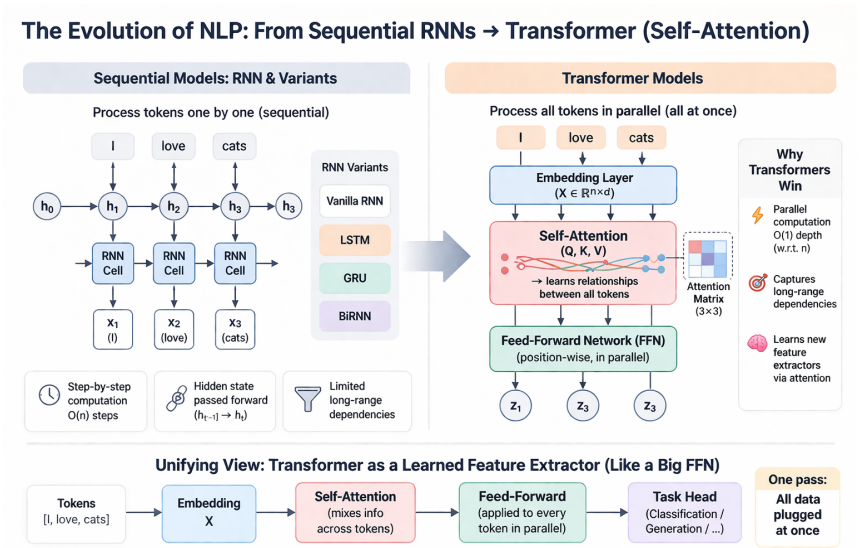
Spring 2025



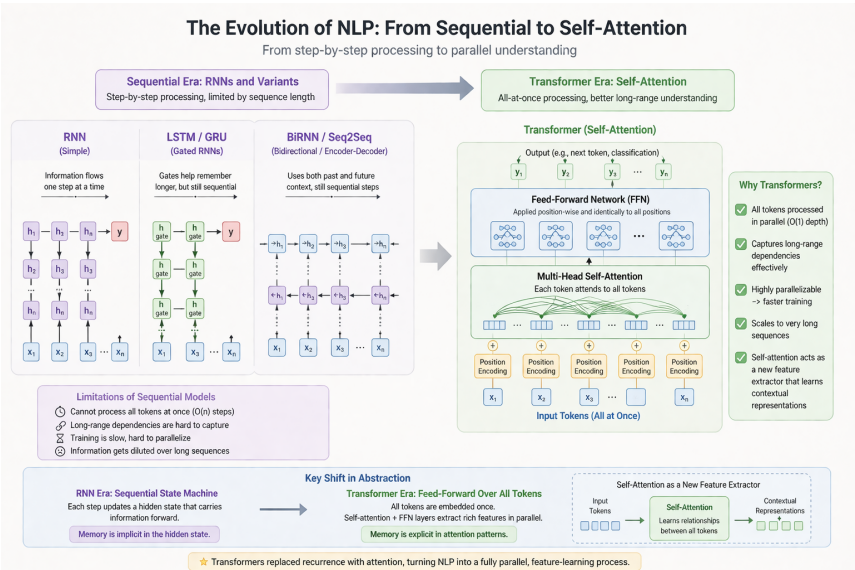
Outline

1. Bridge from Lecture 4: Why Transformers?
2. Positional Encoding
3. Projection as Knowledge Re-Representation
4. Self-Attention: The Core Mechanism
5. Masked Self-Attention
6. Complexity of Self-Attention
7. Summary and Preview

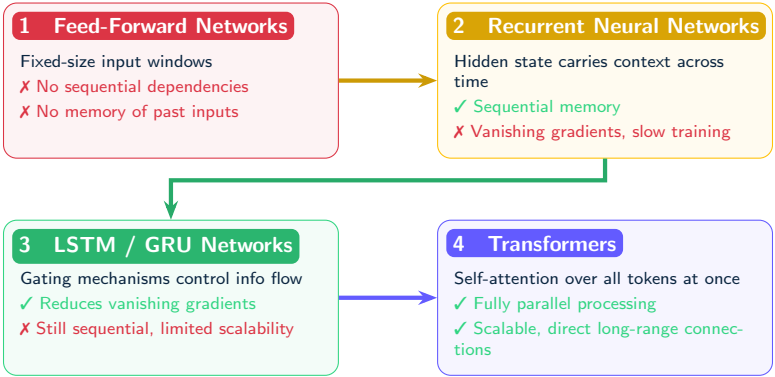
The Evolution of NLP: From Sequential RNNs to Transformers



From Sequential to Self-Attention: A Detailed View



Recap: The Evolution of Language Models



No memory → Sequential memory → Gated memory → **Parallel attention**

Three Limitations of RNN/LSTM

1. Encoding Bottleneck

Entire sequence compressed into one fixed-size vector

2. No Parallelisation

h_t requires h_{t-1}

tokens processed one-by-one

3. Limited Long-Range

Information decays over many time steps

The Transformer Solution

Replace recurrence with **self-attention**:

Problem	RNN	Transformer
Bottleneck	Single $\boldsymbol{h}_T$	Per-token repr.
Parallel	Sequential	All at once
Range	$O(n)$ hops	$O(1)$ direct

Key Paper

Vaswani et al., 2017

"Attention is All You Need"

Why Transformers? Advantage 1: Parallel Processing

✗ RNNs / LSTMs

Process tokens **sequentially**: $h_1 \to h_2 \to h_3 \to \ldots$

Must finish token t before starting $t{+}1$.

Training time scales **linearly** with sequence length.



Must wait for previous step

✓ Transformers

Process **all tokens simultaneously** via matrix multiplication.

Self-attention computes all pairwise relationships in **one step**.

Leverages massive **GPU parallelism** — much faster training.



All tokens processed at once

Result: Training a Transformer on a 1000-token sequence is **orders of magnitude faster** than an RNN, because all 1000 tokens are processed in parallel rather than one-by-one.

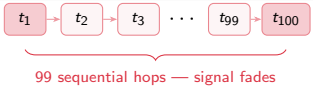
Why Transformers? Advantage 2: Long-Range Connections

✗ RNNs / LSTMs

Information must travel through **every intermediate step**.

Token 1 and token 100 are **99 hops apart**.

Signal **degrades** over distance (vanishing gradients).

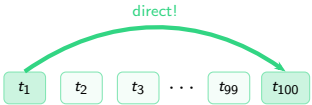


✓ Transformers

Direct connection between any two tokens via attention.

Token 1 and token 100 are **1 hop apart**.

No information degradation — **O(1) path length**.



	RNN path length	Transformer path length
Adjacent tokens	$O(1)$	$O(1)$
Tokens k apart	$O(k)$	$O(1)$
Worst case (n tokens)	$O(n)$	$O(1)$

Why Transformers? Advantage 3: Dynamic Context

✗ RNNs / LSTMs

Context compressed into a **fixed-size hidden state** $\boldsymbol{h}_t \in \mathbb{R}^d$.

All past information squeezed into one vector — **information bottleneck**.

Cannot selectively focus on relevant past tokens.

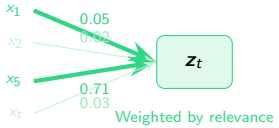


✓ Transformers

Dynamic attention across the entire sequence.

Each token **selectively focuses** on the most relevant other tokens.

Weights are **content-dependent** — different queries attend differently.

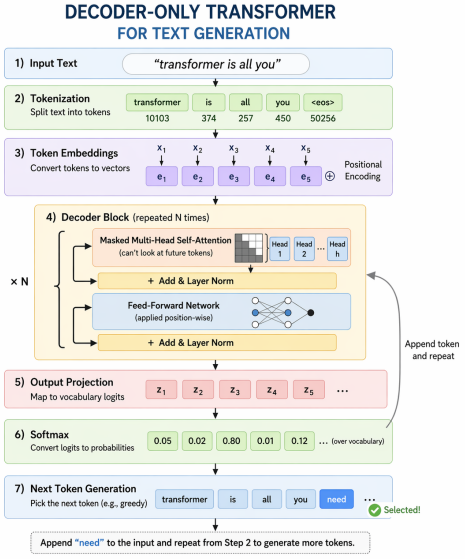


Impact: Transformers enabled breakthrough models like **BERT**, **GPT**, and **T5**, revolutionising NLP across understanding, generation, and translation tasks.

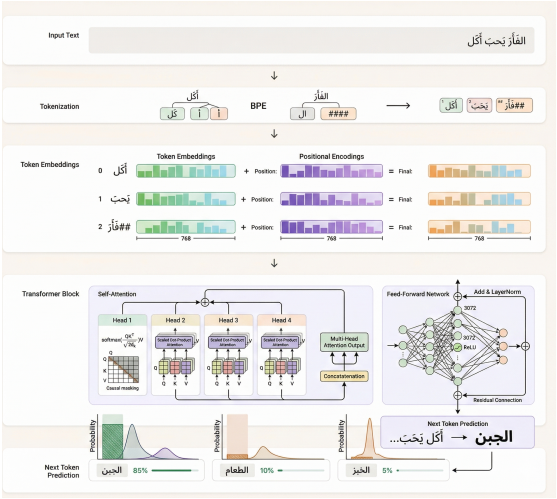
Outline

1. Bridge from Lecture 4: Why Transformers?
2. Positional Encoding
3. Projection as Knowledge Re-Representation
4. Self-Attention: The Core Mechanism
5. Masked Self-Attention
6. Complexity of Self-Attention
7. Summary and Preview

Decoder-Only Transformer: The Big Picture



Inside the Transformer Block



5 Steps Inside Each Block

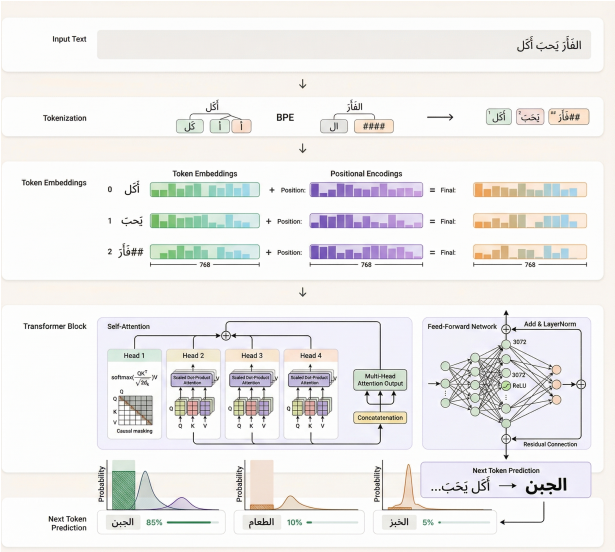
Think of each block as an assembly line — data flows top to bottom:

1. **Attention** — each word "looks around" to decide which other words matter most for its meaning
2. **Stabilise** — keep a copy of the original signal and smooth things out (prevents information loss)
3. **Think Deeper** — each word is independently enriched through a small neural network
4. **Stabilise Again** — same smoothing trick, applied a second time
5. **Predict** — convert the enriched representation into a probability over every word in the vocabulary

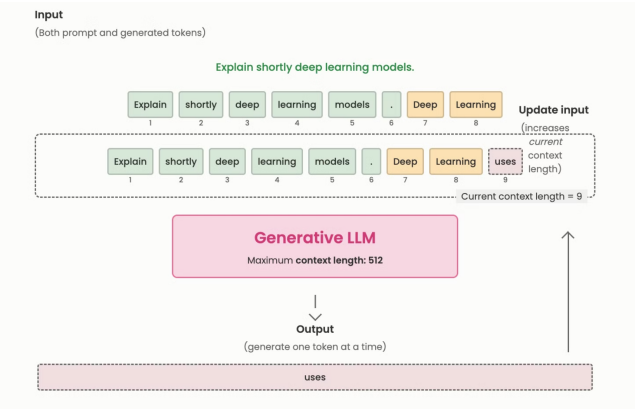
Why Stack Many Blocks?

Each block captures richer patterns. BERT uses 12 blocks; GPT-3 uses 96. More blocks = deeper understanding.

The Generative Transformer: Full Data Flow



Token Generation: Autoregressive Process



How Generation Works

1. Start with a **prompt**
2. Process through all Transformer blocks
3. Get probability distribution over vocab
4. Select next token (e.g. "uses")
5. **Append** to input and repeat

Key Properties

Autoregressive: one token at a time
Context length: max tokens (512, 8K, 128K)
Causal masking: sees only past tokens

What Does Autoregressive Mean?

Intuition

Imagine writing a sentence **one word at a time**:

1. You write "The"
2. Given "The", you pick "cat"
3. Given "The cat", you pick "is"
4. Given "The cat is", you pick "sleeping"

Each word choice depends on **all previous words**, but you **never peek ahead**.

Key Insight

The model generates **left-to-right**, one token at a time. The full sequence is built by **chaining** conditional predictions.

Mathematical Definition

The joint probability of a sequence factorizes as:

$$P(x_1, \ldots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_{<t})$$

Expanded for "The cat is sleeping":

$$\begin{aligned} & P(\text{The}) \\ \times\ & P(\text{cat} \mid \text{The}) \\ \times\ & P(\text{is} \mid \text{The, cat}) \\ \times\ & P(\text{sleeping} \mid \text{The, cat, is}) \end{aligned}$$

Models: GPT, LLaMA, Mistral, Claude

Representational Models: Not Autoregressive

Intuition

Instead of predicting left-to-right, the model sees the **entire sentence** with some words **masked out**, and predicts the missing words:

1. Input: "The [MASK] is sleeping"
2. The model sees **both** left and right context
3. It predicts [MASK] = "cat" using **all** surrounding words

This is **bidirectional** — the model looks in **both directions** simultaneously.

Key Difference

Generative (GPT): left-to-right, one token at a time
Representational (BERT): sees full context, fills in blanks

Masked Language Model (MLM) Objective

Given a set of masked positions $\mathcal{M}$, the training objective is:

$$\mathcal{L}_{\text{MLM}} = -\sum_{t \in \mathcal{M}} \log P(x_t \mid \mathbf{x}_{\setminus \mathcal{M}})$$

Each masked token is predicted from **all unmasked tokens** $\mathbf{x}_{\setminus \mathcal{M}}$.

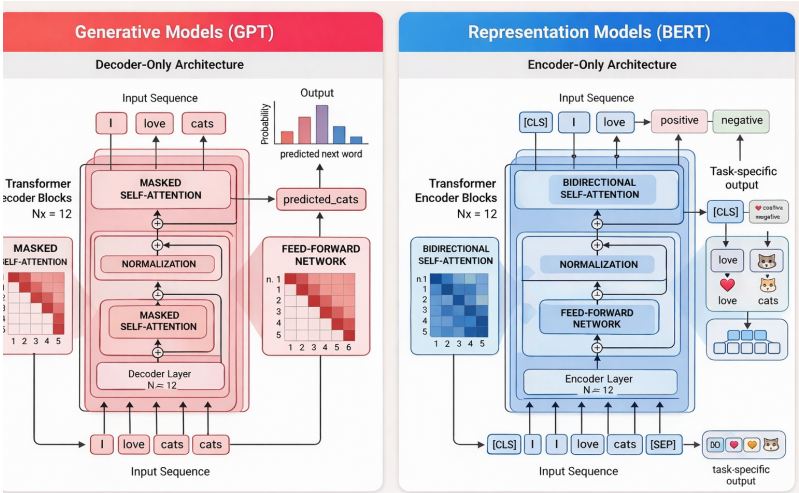
Expanded for "The [MASK] is sleeping":

$$P(\text{cat} \mid \text{The},\ \text{is},\ \text{sleeping})$$

No chain rule — predictions are **independent** given context.

Models: BERT, RoBERTa, ELECTRA, DeBERTa

Two Families: Generative vs. Representational



Two Families — What to Notice

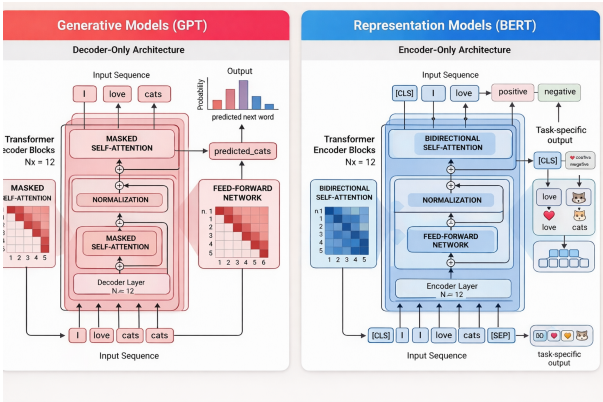
Same building blocks, different visibility rules:

GPT — Generative

- Words only see what came **before** them (red triangular mask in the diagram)
- Like reading left-to-right **with blinders on**
- Goal: predict the **next word**

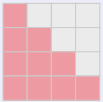
BERT — Representational

- Every word sees **all** other words at once (blue full grid in the diagram)
- Like reading the **entire page** before answering
- Goal: build a deep **understanding** of each word



Generative vs. Representational: Key Differences

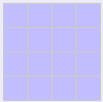
Generative (Decoder-Only)



Causal mask

- **Masked** attention — sees only past tokens
- **Autoregressive** — one token at a time
- Left-to-right processing
- **Ex:** GPT, LLaMA, Mistral, Claude
- **Tasks:** generation, chat, code

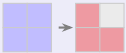
Representational (Encoder-Only)



Full attention

- **Bidirectional** — sees all tokens at once
- **Contextual embeddings** as output
- Special tokens: `[CLS]`, `[SEP]`
- **Ex:** BERT, RoBERTa, ALBERT
- **Tasks:** classify, NER, QA

Hybrid (Encoder-Decoder)

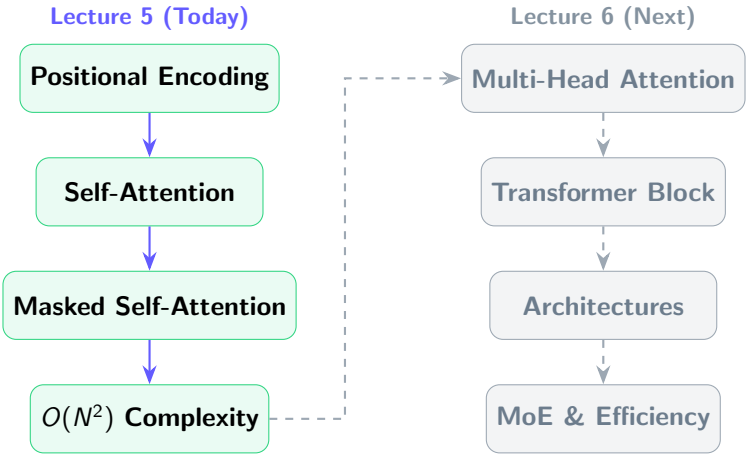


Both patterns

- **Encoder**: full attention on input
- **Decoder**: causal + cross-attention
- Seq-to-seq architecture
- **Ex:** T5, BART, mBART
- **Tasks:** translation, summarisation

Today's focus: The **self-attention mechanism** that powers *all three* families.
The key difference is the **attention mask**: causal vs. full .

Road Map for This Lecture



Outline

1. Bridge from Lecture 4: Why Transformers?
2. Positional Encoding
3. Projection as Knowledge Re-Representation
4. Self-Attention: The Core Mechanism
5. Masked Self-Attention
6. Complexity of Self-Attention
7. Summary and Preview

The Problem: Transformers Have No Sense of Order

The Issue

Unlike RNNs that process tokens **sequentially**, Transformers process all tokens **simultaneously**. They have **no inherent sense of word order**.

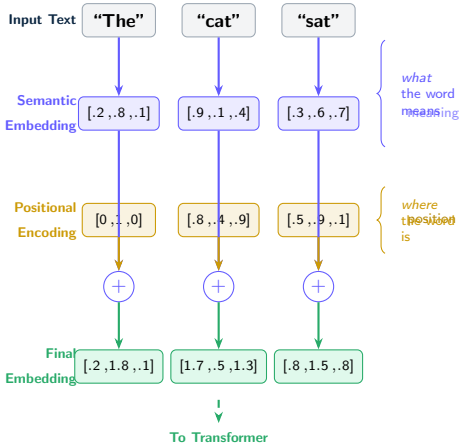
Consider these two sentences:

- "The **cat** sat on the **mat**"
- "The **mat** sat on the **cat**"

Without position information, a Transformer produces **identical representations** for both!

Solution

Add a **positional encoding** vector to each token embedding **before** it enters the Transformer.



How Positional Encoding Works

Token Embeddings
Meaning of each word

+

Position Encodings
Order information

=

Position-Aware Embeddings
Meaning + Order

Example: "The cat chased the dog"

Token Emb.	The	cat	chased	the	dog
	+	+	+	+	+
Position Enc.	PE_0	PE_1	PE_2	PE_3	PE_4
	=	=	=	=	=
Final Input	The+PE_0	cat+PE_1	chased+PE_2	the+PE_3	dog+PE_4

✓ **Preserves Word Order**
Enables understanding of grammar and meaning

✓ **Maintains Parallelism**
Added via simple summation — no sequential processing

✓ **Variable Lengths**
Works for any sequence length without architecture changes

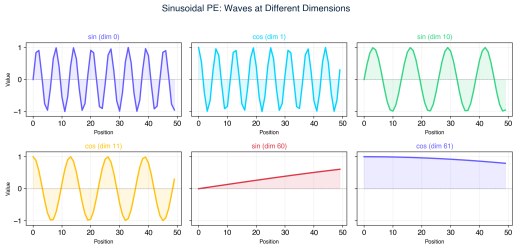
Sinusoidal Positional Encoding: The Formula

The Encoding Formula

For position pos and dimension index i:

$$\mathrm{PE}(\mathrm{pos}, 2i) = \sin\left(\frac{\mathrm{pos}}{10000^{2i/d_{\mathrm{model}}}}\right)$$

$$\mathrm{PE}(\mathrm{pos}, 2i{+}1) = \cos\left(\frac{\mathrm{pos}}{10000^{2i/d_{\mathrm{model}}}}\right)$$



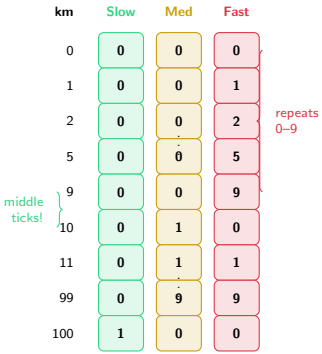
- **Even** dimensions → sine
- **Odd** dimensions → cosine
- Each dimension oscillates at a **different frequency**
- Lower dims change fast, higher dims change slow

Positional Encoding: The Odometer Analogy

Think of a Car Odometer

An odometer has multiple **wheels**, each spinning at a different speed:

- **Rightmost wheel**: changes every km (fast — like dim 0)
- **Middle wheel**: changes every 10 km (medium — like dim 1)
- **Leftmost wheel**: changes every 100 km (slow — like dim 2)



Same Idea with Sine Waves

Replace digits with **sine/cosine waves** at different speeds:

- **Dim 0**: wave completes a full cycle every ∼6 positions (fast)
- **Dim 2**: wave completes a cycle every ∼628 positions (slow)

"Frequency" = how fast the wave oscillates.
High frequency → fast change. Low frequency → slow change.

Why Not Just Use pos $= 0, 1, 2, \ldots$?

- Pos 500 would be **500**× **larger** than pos 1 — **unstable**
- No notion of "closeness" between positions

Why Sine/Cosine Work

- Always between -1 and $+1$ — **bounded & stable**
- Nearby positions get **similar** values
- The **combination** of all dims is unique per position
- Works for **any** sequence length (no retraining)

Numerical Example: $d_{\text{model}} = 4$, Positions 0–7

Setup: $d_{\text{model}} = 4$ gives 2 frequency pairs ($i = 0$ and $i = 1$):

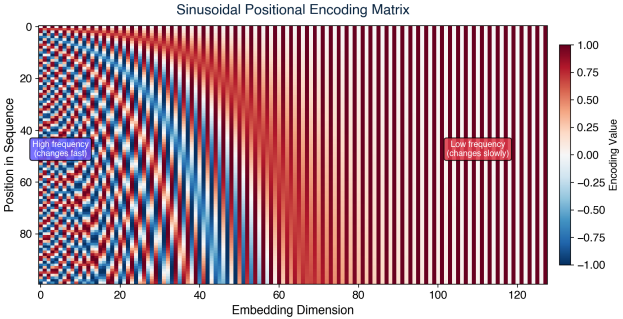
Dim	Frequency (wavelength)
i=0 (dims 0,1)	$\omega = 1/10000^{0/4} = 1.0$ period $= 2\pi \approx 6.3$ **fast cycle**
i=1 (dims 2,3)	$\omega = 1/10000^{2/4} = 0.01$ period $= 200\pi \approx 628$ **slow cycle**

Computed PE values (rounded to 2 d.p.):

pos	Fast pair (i=0) sin(pos)	Fast pair (i=0) cos(pos)	Slow pair (i=1) sin(.01 · pos)	Slow pair (i=1) cos(.01 · pos)
0	0.00	1.00	0.00	1.00
1	0.84	0.54	0.01	1.00
2	0.91	−0.42	0.02	1.00
3	0.14	−0.99	0.03	1.00
4	−0.76	−0.65	0.04	1.00
5	−0.96	0.28	0.05	1.00
6	−0.28	0.96	0.06	1.00
7	0.66	0.75	0.07	1.00

Key observation: The fast pair cycles and **repeats** (pos 0 ≈ pos 6) — but the slow pair keeps changing, so the **full 4D vector stays unique**.
Like a clock: the seconds hand repeats, but minutes + hours make each moment distinct.

Visualising Sinusoidal Positional Encoding



Each row = one position, each column = one dimension.
Colour = PE value (-1 to $+1$).

Reading the Heatmap

- Left columns: high frequency (fast oscillation)
- Right columns: low frequency (slow change)
- Every row is **unique** — each position gets a distinct "fingerprint"

Why This Matters

The model can learn to distinguish positions by combining signals at multiple frequencies — like a **Fourier encoding** of position.

Why Sine and Cosine?

Unique Positions

Every position gets a unique encoding vector. The model can distinguish position 5 from position 50.

Captures Patterns

Wave-like structures make it easy for the model to learn *relative* positions: "3 tokens apart" = constant.

Extrapolation

Works for sequences longer than training data. The formula computes PE for *any* position.

Key Property: $\text{PE}(\text{pos} + k)$ can be expressed as a **linear function** of $\text{PE}(\text{pos})$, allowing the model to learn relative position relationships.

Sinusoidal PE: What's Good and What's Not

✓ Advantages

Understands "How Far Apart"

The wave patterns let the model figure out that two words are e.g. 3 positions apart.

Works for Any Sentence Length

Since it's a math formula, it can compute a position code for pos 10 or pos 10,000.

Nothing to Learn

No extra trainable weights needed — just plug in the formula and go.

✗ Limitations

Cannot Adapt to the Task

The formula is fixed — the model can't learn better position patterns from data.

Struggles with Very Long Texts

For extremely long sequences (e.g. 100K+), the patterns may become less reliable.

One-Size-Fits-All Frequencies

The wave speeds are preset — they may not be ideal for every type of text.

In practice: Sinusoidal PE was the original approach (2017). Modern models like **BERT** use **learned embeddings** and **LLaMA**/**GPT-4** use **RoPE** (Rotary Position Embeddings) — which are learnable and handle long texts better.

Learnable Positional Embeddings

What Are They?

Instead of a fixed formula, the model **learns** a position embedding during training — just like word embeddings.

How They Work:

1. Create a **lookup table** with one vector per position
2. Initialise randomly; training adjusts them via backpropagation
3. Added to word embeddings: $\boldsymbol{x}_t = \boldsymbol{e}_t + \boldsymbol{p}_t$

Sinusoidal = GPS coordinates (fixed formula, works everywhere)
Learned = each seat picks its own name tag (adapts to the class)

✓ Advantages

- **Flexible**: adapts to task and data
- **Optimised**: model decides what position info matters

✗ Limitations

- **No extrapolation**: can't handle positions beyond training length
- **Extra parameters**: adds $L \times d$ trainable weights

Used in

BERT, GPT-2/3, T5 — most modern Transformers

Positional Encoding vs. Positional Embedding

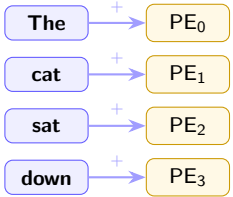
	Sinusoidal Encoding	Learned Embedding
What is it?	A **fixed formula** computes position vectors — no training needed	A **lookup table** of vectors that the model learns during training
New lengths?	Yes — the formula works for any position, even ones never seen in training	No — if trained on 512 tokens, position 513 has no vector
Adapts to data?	No — same pattern for every task	Yes — the model discovers what position patterns matter for its task
Extra cost	Free — computed on the fly	Adds $L \times d$ trainable parameters
Used in	Original Transformer (2017)	BERT, GPT-2/3, T5
When to pick?	You need to handle **sequences longer** than training data	You have **enough training data** and a fixed maximum length is OK

Key trade-off: Works on any length (but rigid) vs. Adapts to the task (but can't go beyond training length)

Absolute Positional Encoding (APE)

The Idea

Each position in the sequence gets its own **unique vector** — like an ID card. Position 0 always has the same vector, position 1 always has the same vector, and so on.



How it works:

Create a table of vectors, one per position. Look up the vector for each position and **add** it to the token embedding:

$$\boldsymbol{x}_t = \boldsymbol{e}_t + \boldsymbol{p}_t \qquad \text{where } \mathbf{P} \in \mathbb{R}^{L \times d}$$

This is what sinusoidal PE and learned PE both do — they only differ in *how* the table is filled (formula vs. training).

Used in

BERT, GPT-2, ALBERT, ViT

Limitation

Position 513 during inference? If the model only trained up to 512 — **no vector exists**. APE cannot extrapolate.

PyTorch Lab: APE — Learned Position Embeddings

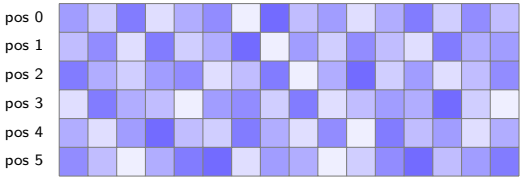
Create a learnable lookup table of position vectors. Visualise how each position gets its own unique pattern.

```python
import torch, torch.nn as nn, matplotlib.pyplot as plt
max_len, d_model = 20, 16
pos_embedding = nn.Embedding(max_len, d_model) # lookup table
positions = torch.arange(6) # 6-word sentence
pe_vectors = pos_embedding(positions) # shape: (6, 16)
print("Pos 0:", pe_vectors[0].data.round(decimals=2))
print("Pos 1:", pe_vectors[1].data.round(decimals=2))
# Plot: each row = one position's embedding vector
plt.figure(figsize=(10, 3))
plt.imshow(pe_vectors.detach(), aspect='auto', cmap='RdBu')
plt.xlabel("Embedding dimension"); plt.ylabel("Position")
plt.yticks(range(6), [f"pos {i}" for i in range(6)])
plt.title("APE: Each position has a unique learned pattern")
plt.colorbar(label="Value"); plt.tight_layout(); plt.show()
# Beyond training length? --> CRASH
try: pos_embedding(torch.tensor([25])) # pos 25 > max_len=20
except: print("ERROR: position 25 not in table! No extrapolation.")
```

Run this and see: each position gets a unique colour pattern (random at init, refined during training). The last line shows APE's limitation: requesting a position beyond the table size **crashes** — no extrapolation.

APE Expected Output: Each Position Gets a Unique Pattern

Heatmap: Learned Position Embeddings



Each row has a **unique irregular pattern** — these are random at initialisation, then refined during training.

Console Output

```
Pos 0:  [ 0.12 -0.43  0.78 -0.21 ...]
Pos 1:  [-0.55  0.31  0.14  0.67 ...]
Shape:  torch.Size([6, 16])

ERROR: position 25 not in table!
No extrapolation.
```

Key observations:

✓ Each position has a **unique vector**

✓ Patterns are **learned** (not formulaic)

✗ Position 25 $>$ table size 20 $\rightarrow$ **crash**

This is why APE cannot handle sequences longer than training length.

Learned PE: Try It Yourself

Setup: max_len $= 4$, $d = 3$, sentence $=$ "The cat sat down"

Lookup table (initialised randomly):

$$\mathbf{P} = \begin{bmatrix} 0.12 & -0.43 & 0.78 \\ 0.55 & 0.31 & -0.14 \\ -0.22 & 0.67 & 0.41 \\ 0.38 & -0.19 & 0.56 \end{bmatrix} \quad \leftarrow \text{rows} = \text{pos } 0,\ 1,\ 2,\ 3$$

Token embeddings: "The" $= [0.2, -0.1, 0.4]$, "cat" $= [0.8, 0.6, 0.3]$

Your task:

1. What is the position vector for "cat" (position 1)?
2. Compute the input representation: $\boldsymbol{x}_{\text{cat}} = \boldsymbol{e}_{\text{cat}} + \boldsymbol{p}_1 = ?$
3. What happens if you try to look up position 5?

It's just element-wise addition — try it on paper...

Learned PE: Solution

Full computation for all 4 tokens:

Token	Embedding e_t	+	Position p_t	= Input x_t
"The" (pos 0)	$[0.2, -0.1, 0.4]$	+	$[0.12, -0.43, 0.78]$	= **[0.32, -0.53, 1.18]**
"cat" (pos 1)	$[0.8, 0.6, 0.3]$	+	$[0.55, 0.31, -0.14]$	= **[1.35, 0.91, 0.16]**
"sat" (pos 2)	$[0.5, 0.4, 0.2]$	+	$[-0.22, 0.67, 0.41]$	= **[0.28, 1.07, 0.61]**
"down" (pos 3)	$[0.3, 0.1, 0.7]$	+	$[0.38, -0.19, 0.56]$	= **[0.68, -0.09, 1.26]**

Answer to Q3: Position 5 is **not in the table** (max_len $= 4$) $\rightarrow$ crash!

Training adjusts **P** via backpropagation so positions in similar contexts get similar vectors. But **no formula exists to compute new positions** — learned PE cannot extrapolate beyond the table.

Relative Positional Encoding (RPE)

The Problem APE Doesn't Solve

Consider: "The **cat** sat on the **mat**."

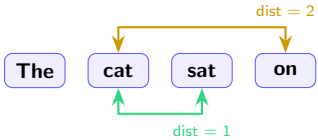
APE tells us *cat* is at position 1 and *mat* is at position 5. But what matters for understanding is that *mat* is **4 words away** from *cat* — the **distance**, not the absolute positions.

RPE solution: Instead of encoding "I am at position 5", encode "this token is **3 positions to my right**."

How: When computing attention between positions i and j, add a **bias term** b_{i-j} that depends only on the distance:

$$\text{score}(i, j) = \boldsymbol{q}_i \cdot \boldsymbol{k}_j + b_{i-j}$$

The bias b_{i-j} is learned (one scalar per distance value).



Key advantage:
"cat" and "sat" are 1 apart **regardless of where they appear** in the sentence. The model learns that distance-1 means something, not that position-1-and-2 means something.

Used in

T5, Transformer-XL, DeBERTa

Limitation

Needs a bias for every possible distance — adds complexity, and distances must be clipped for very long sequences.

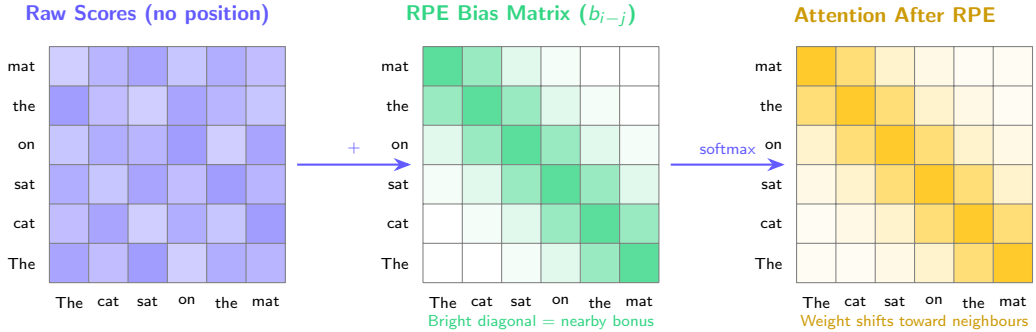
PyTorch Lab: RPE — Distance-Based Attention Bias

Build a relative bias matrix and see how it changes attention: nearby tokens get a bonus, distant ones get a penalty.

```python
import torch, torch.nn.functional as F, matplotlib.pyplot as plt
seq_len = 6
tokens = ["The", "cat", "sat", "on", "the", "mat"]
# RPE: one learned bias per distance. Nearby = bonus, far = penalty
bias_values = {0: 0.5, 1: 0.3, 2: 0.0, 3: -0.1, 4: -0.2, 5: -0.3}
bias_matrix = torch.zeros(seq_len, seq_len)
for i in range(seq_len):
    for j in range(seq_len):
        bias_matrix[i, j] = bias_values.get(abs(i-j), -0.3)
# Add bias to raw scores: score(i,j) = q_i . k_j + b_{i-j}
raw_scores = torch.randn(seq_len, seq_len) * 0.3
rpe_scores = raw_scores + bias_matrix
weights = F.softmax(rpe_scores, dim=-1)
# Plot 3 heatmaps side by side
fig, axes = plt.subplots(1, 3, figsize=(14, 4))
for ax, data, t in zip(axes, [raw_scores, bias_matrix, weights],
    ["Raw scores (no position)", "RPE bias matrix", "After RPE"]):
    ax.imshow(data.detach(), cmap='Blues')
    ax.set_xticks(range(6)); ax.set_xticklabels(tokens, fontsize=8)
    ax.set_yticks(range(6)); ax.set_yticklabels(tokens, fontsize=8)
    ax.set_title(t)
plt.tight_layout(); plt.show()
```

Run this and see 3 heatmaps: (1) raw scores ignore position, (2) the bias matrix adds a distance-dependent bonus, (3) final attention shifts weight toward **nearby** tokens. The bias b_{i-j} is the same whether "cat–sat" appears at positions 1–2 or 10–11.

RPE Expected Output: How Distance Bias Changes Attention



Left: Without position info, attention is scattered. **Middle:** The RPE bias matrix adds a bonus for nearby tokens (bright diagonal) and a penalty for distant ones. **Right:** After adding the bias, attention concentrates on **neighbouring tokens**.

RPE: Try It Yourself

Sentence: "The cat sat" **Learned biases:** $b_0 = 0.5$, $b_1 = 0.3$, $b_2 = 0.0$

Raw attention scores (from QK^T, no position info yet):

$$S = \begin{bmatrix} 0.4 & 0.1 & 0.2 \\ 0.3 & 0.5 & 0.1 \\ 0.2 & 0.3 & 0.6 \end{bmatrix}$$

Your task:

1. Build the bias matrix B where $B_{ij} = b_{|i-j|}$
   (distance 0 $\rightarrow$ 0.5, distance 1 $\rightarrow$ 0.3, distance 2 $\rightarrow$ 0.0)
2. Compute $S + B$ (element-wise addition)
3. Which positions got the biggest boost? Why?

Hint: the bias matrix is symmetric and has the largest values on the diagonal...

RPE: Solution

Step 1: Bias matrix from $b_0 = 0.5$, $b_1 = 0.3$, $b_2 = 0.0$: $B = \begin{bmatrix} 0.5 & 0.3 & 0.0 \\ 0.3 & 0.5 & 0.3 \\ 0.0 & 0.3 & 0.5 \end{bmatrix}$

Step 2: Add bias to raw scores:

$$S + B = \begin{bmatrix} 0.4+0.5 & 0.1+0.3 & 0.2+0.0 \\ 0.3+0.3 & 0.5+0.5 & 0.1+0.3 \\ 0.2+0.0 & 0.3+0.3 & 0.6+0.5 \end{bmatrix} = \begin{bmatrix} \mathbf{0.9} & 0.4 & 0.2 \\ 0.6 & \mathbf{1.0} & 0.4 \\ 0.2 & 0.6 & \mathbf{1.1} \end{bmatrix}$$

Step 3: Softmax $\rightarrow \begin{bmatrix} \mathbf{0.50} & 0.30 & 0.20 \\ 0.24 & \mathbf{0.46} & 0.30 \\ 0.15 & 0.27 & \mathbf{0.58} \end{bmatrix}$

Diagonal boosted: $b_0 = 0.5$ rewards self-attention, $b_1 = 0.3$ rewards neighbours. Each token now attends more to **itself and adjacent words**. Same biases work at any position in the sentence.

Rotary Positional Encoding (RoPE)

The Clever Trick

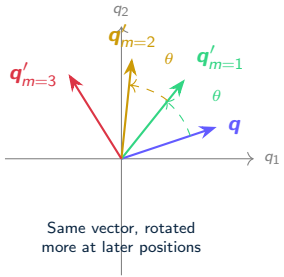
Instead of *adding* a position vector, **rotate** the Query and Key vectors by an angle that depends on their position.

Core idea: Rotate each dimension pair by angle $\theta \cdot \text{pos}$:

$$\begin{bmatrix} q_1' \\ q_2' \end{bmatrix} = \begin{bmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{bmatrix} \begin{bmatrix} q_1 \\ q_2 \end{bmatrix}$$

where m = position index, θ = frequency for this dimension pair.

Why this is brilliant: $\boldsymbol{q}_m \cdot \boldsymbol{k}_n$ depends on $(m - n)$ — the **relative distance** — automatically! Absolute position (each vector rotated by its own angle) + relative position (dot product encodes distance) in one operation.



Same vector, rotated more at later positions

Used in

GPT-4, LLaMA 1/2/3, Mistral, Gemma

✓ No extra parameters
✓ Encodes both absolute & relative
✓ Scales to very long sequences

PyTorch Lab: RoPE — Rotation in Action

Rotate a 2D vector by position-dependent angles. Plot how the dot product depends only on relative distance.

```python
import torch, matplotlib.pyplot as plt

def rotate(v, pos, theta=0.5):
    """Rotate 2D vector v by angle = pos * theta."""
    angle = pos * theta
    R = torch.tensor([[torch.cos(angle), -torch.sin(angle)],
                      [torch.sin(angle), torch.cos(angle)]])
    return R @ v

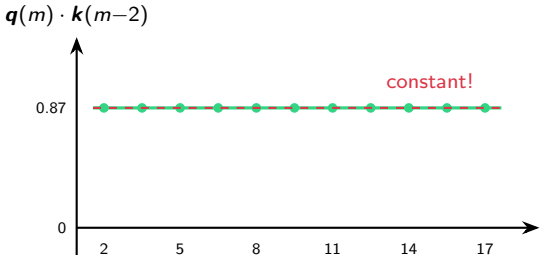
q = torch.tensor([1.0, 0.3]) # original query vector
k = torch.tensor([0.8, 0.5]) # original key vector
positions = range(20)

# Rotate q at position m, k at position n -> compute dot product
# Key insight: dot product depends on (m - n), not on m or n alone
dots_same_dist = [] # fix distance=2, vary absolute positions
for m in range(2, 20):
    dots_same_dist.append((rotate(q, m) @ rotate(k, m-2)).item())

plt.plot(dots_same_dist, 'o-')
plt.title("q(m) . k(m-2): same distance=2, different positions")
plt.xlabel("Starting position m"); plt.ylabel("Dot product")
plt.axhline(y=dots_same_dist[0], color='r', ls='--', label='constant!')
plt.legend(); plt.show() # dot product is CONSTANT -- only distance matters
```

Run this and see: the dot product is **constant** across all positions — it only depends on the **distance** (2). This is why RoPE encodes relative position automatically: rotation makes $\boldsymbol{q}_m \cdot \boldsymbol{k}_n = f(m-n)$.

RoPE Expected Output: Dot Product Is Constant



The dot product $\boldsymbol{q}(m) \cdot \boldsymbol{k}(m-2)$ is the **same value** at every position — it depends only on the **distance** (2), not on where in the sequence.

Why This Matters

- RoPE rotates Q at position m by angle $m\theta$
- RoPE rotates K at position n by angle $n\theta$
- The dot product only sees the **angle difference**: $(m-n)\theta$

$\rightarrow$ **Relative distance is encoded automatically**

Compare with APE:

APE dot products *vary* with position.
RoPE dot products are *constant* at fixed distance.

This is why RoPE handles long sequences better.

RoPE: Try It Yourself

Setup: $\boldsymbol{q} = [1.0,\ 0.3]$, $\boldsymbol{k} = [0.8,\ 0.5]$, $\theta = 0.5$ rad

Recall the rotation formula:

$$\boldsymbol{q}'_m = \begin{bmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{bmatrix} \begin{bmatrix} q_1 \\ q_2 \end{bmatrix}$$

Your task:

1. Rotate $\boldsymbol{q}$ at position $m = 2$ (angle $= 2 \times 0.5 = 1.0$ rad)
   Hint: $\cos(1.0) \approx 0.54$, $\sin(1.0) \approx 0.84$
2. Rotate $\boldsymbol{k}$ at position $n = 0$ (what happens when angle $= 0$?)
3. Compute $\boldsymbol{q}'_2 \cdot \boldsymbol{k}'_0$
4. **Predict:** if you try $m = 5, n = 3$ instead (same distance $= 2$), will you get a different answer?

Work through it on paper, then we'll check...

RoPE: Solution

Step 1: Rotate Q at position $m = 2$ (angle $= 1.0$ rad)

$$\boldsymbol{q}_2' = \begin{bmatrix} 0.54 & -0.84 \\ 0.84 & 0.54 \end{bmatrix} \begin{bmatrix} 1.0 \\ 0.3 \end{bmatrix} = \begin{bmatrix} 0.54 - 0.25 \\ 0.84 + 0.16 \end{bmatrix} = \begin{bmatrix} \mathbf{0.29} \\ \mathbf{1.00} \end{bmatrix}$$

Step 2: Rotate K at position $n = 0$ (angle $= 0 \rightarrow$ identity matrix, no change)

$$\boldsymbol{k}_0' = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0.8 \\ 0.5 \end{bmatrix} = \begin{bmatrix} \mathbf{0.80} \\ \mathbf{0.50} \end{bmatrix}$$

Step 3: Dot product $\boldsymbol{q}_2' \cdot \boldsymbol{k}_0' = 0.29 \times 0.80 + 1.00 \times 0.50 = \mathbf{0.73}$

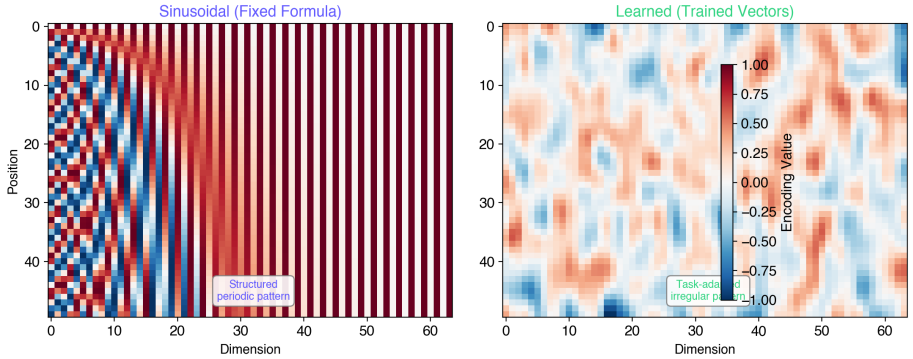
Answer to Q4: Yes — $m = 5, n = 3$ gives **the same 0.73**. The dot product depends only on the **distance** $(m - n = 2)$, not the absolute positions. That's the magic of RoPE.

PE Variants: Summary and Comparison

	APE	RPE	RoPE
What it encodes	"I am at position 5"	"He is 3 words away"	Both at once
How	Add a vector to each token	Add bias b_{i-j} to scores	Rotate Q and K vectors
Long sequences	Fails beyond training length	Good (distance is bounded)	Excellent (angle extrapolates)
Extra params	$L \times d$	One per distance	Zero
Models	BERT, GPT-2, ViT	T5, DeBERTa	GPT-4, LLaMA, Mistral

Historical trend: APE (2017–2019) $\rightarrow$ RPE (2019–2021) $\rightarrow$ **RoPE** (2021–today). Each generation solved a limitation of the previous one. Almost all modern LLMs now use RoPE.

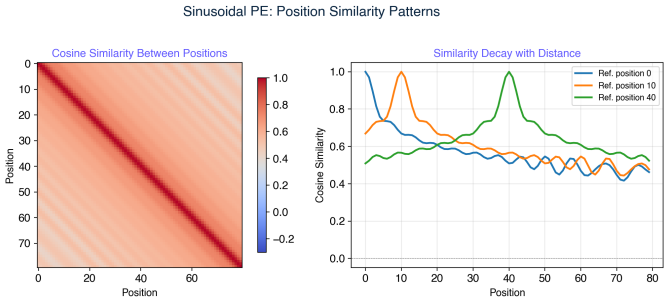
Sinusoidal vs. Learned: Visual Comparison



Left: Sinusoidal PE shows **regular stripes** — the formula produces the same pattern every time.
Right: Learned PE shows **irregular patches** — the model adapted these vectors to fit the training data.

What this tells you: Sinusoidal patterns are predictable and repeat — great for generalisation. Learned patterns are messy but task-specific — the model discovered which positions need similar or different encodings.

How PE Encodes Position Similarity



Heatmaps show **cosine similarity** between every pair of PE vectors: $\text{sim}(\boldsymbol{p}_i, \boldsymbol{p}_j) = \dfrac{\boldsymbol{p}_i \cdot \boldsymbol{p}_j}{\|\boldsymbol{p}_i\| \, \|\boldsymbol{p}_j\|}$

Left: Bright diagonal = nearby positions have similar vectors. **Right:** Similarity decays smoothly with distance.

Key insight: Nearby positions get **similar PE vectors** → the model knows "these words are close together." Words 3 apart always have the same similarity — the Transformer encodes **relative distance** from a purely absolute encoding.

Self-Check Q1

Why can sinusoidal positional encoding generalise to sequences **longer** than those seen during training, while learned positional embeddings **cannot**?

Think about how each approach "knows" a position...

PyTorch Lab: Sinusoidal Positional Encoding

Implement the exact PE formula from this lecture: $PE(pos, 2i) = \sin(pos/10000^{2i/d})$

```python
import torch, math

d_model = 4 # embedding dimension (same as lecture)
max_len = 3 # "I", "like", "AI"

pe = torch.zeros(max_len, d_model)
pos = torch.arange(max_len).unsqueeze(1).float() # (3, 1)

# Denominator: 10000^(2i / d_model) for each dimension pair
div = torch.exp(
    torch.arange(0, d_model, 2).float() * -(math.log(10000.0) / d_model)
)

pe[:, 0::2] = torch.sin(pos * div) # even dimensions: sin
pe[:, 1::2] = torch.cos(pos * div) # odd dimensions: cos

print("PE matrix (3 tokens x 4 dims):")
print(pe.round(decimals=4))
# Row 0 = position 0, Row 1 = position 1, Row 2 = position 2
```

Each row is one position's encoding vector. Low-frequency sin/cos waves in the first dims, high-frequency in the last — exactly the formula from the previous slides.

PyTorch Lab: Embeddings + Positional Encoding

Combine word meaning (X) with word order (PE) — this is the actual input to self-attention.

```python
import torch

# Same toy embeddings as the lecture's worked example
X = torch.tensor([
    [0.2, -0.1, 0.4, 0.1], # "I"
    [0.8, 0.6, 0.3, 0.5], # "like"
    [0.5, 0.4, 0.2, 0.7], # "AI"
])

# pe from previous slide (sinusoidal, shape 3 x 4)
X_pos = X + pe # element-wise addition: meaning + position

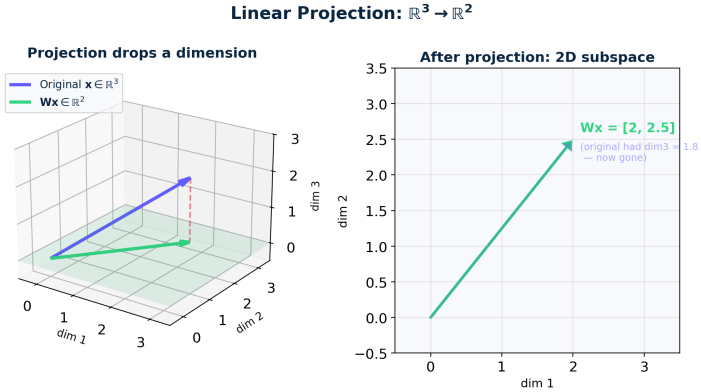
print("Token embeddings X:\n", X)
print("\nPositional encoding PE:\n", pe.round(decimals=4))
print("\nPosition-aware input X + PE:\n", X_pos.round(decimals=4))
```

`X_pos = X + pe` — this is the actual input to self-attention. It carries both **what** the word means and **where** it sits in the sentence. We will use `X_pos` in all coding slides that follow.

Outline

1. Bridge from Lecture 4: Why Transformers?
2. Positional Encoding
3. Projection as Knowledge Re-Representation
4. Self-Attention: The Core Mechanism
5. Masked Self-Attention
6. Complexity of Self-Attention
7. Summary and Preview

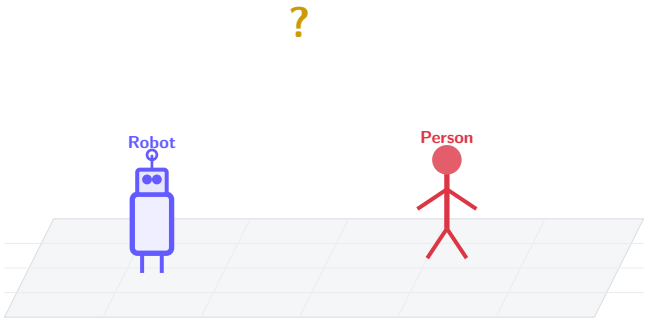
What Does a Linear Projection Do?



Projection = $\mathbf{W}\boldsymbol{x}$: the **same vector** passes through different matrices, each **changing the dimension** and exposing a **different aspect** of the same knowledge.

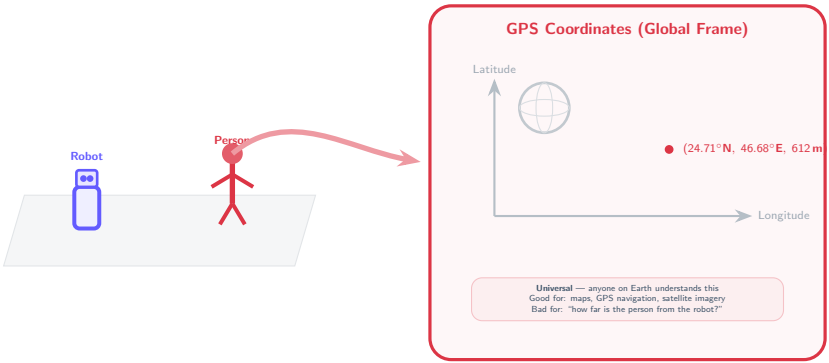
Nothing is created or destroyed — $\mathbf{W}$ is **learned** to select exactly what the task needs, like viewing the same scene through different lenses.

A Simple Question Before We Start...



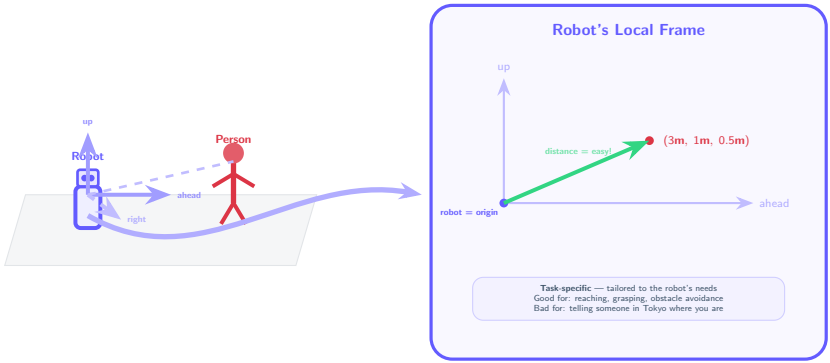
The answer depends on **who is asking** and **what they need**. Keep this in mind...

Answer 1: Ask a Cartographer



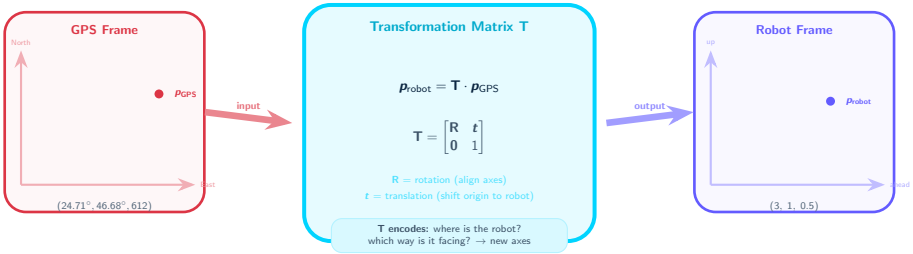
A cartographer gives you the **absolute position** — precise, universal, but **not tailored to any specific task**. This is like a **raw embedding**: it knows everything, but nothing is emphasised.

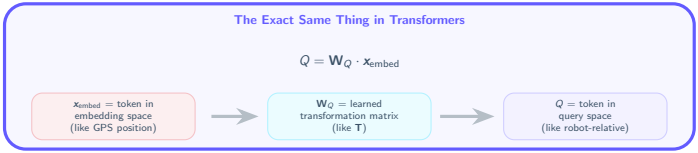
Answer 2: Ask the Robot



The robot gives you a **task-specific** answer — "3 metres ahead, 1 right, 0.5 up". Same person, **same information**, but re-expressed so the **relevant feature** (distance) is trivially readable.

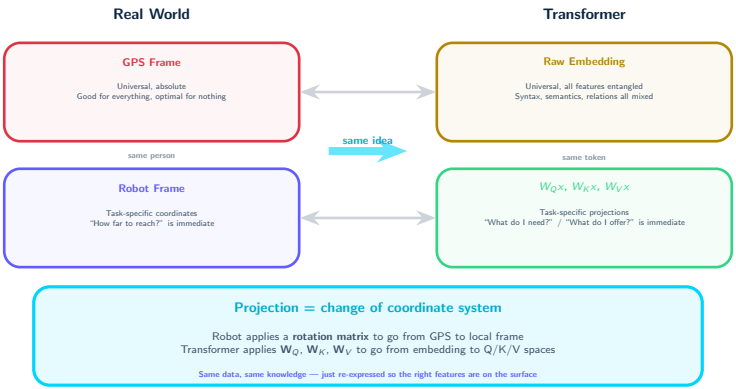
The Transformation Matrix: How to Switch Frames





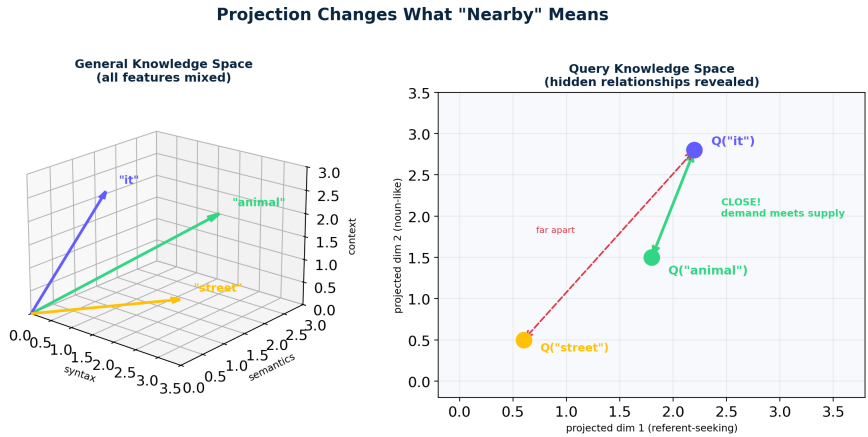
The transformation matrix **T is** the weight matrix **W**. In robotics it is computed from geometry; in transformers it is **learned from data** — but the operation is identical: $\boldsymbol{y} = \mathbf{W}\boldsymbol{x}$.

From Coordinates to Transformers: Same Principle



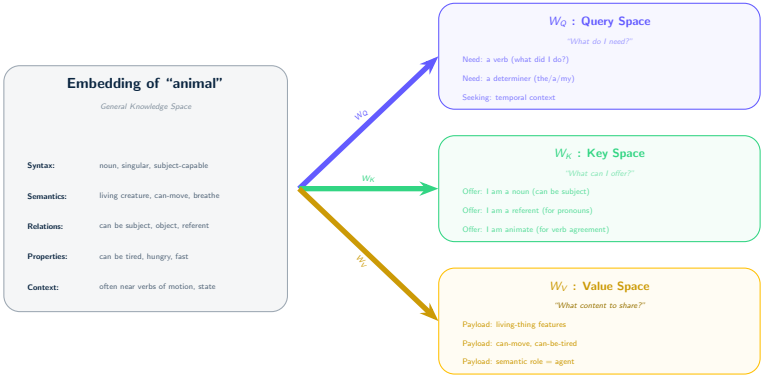
Projection does **not create or destroy** information — it **re-expresses** it in a coordinate system where the *right* features are immediately accessible. This is exactly what W_Q, W_K, W_V do to each token.

Projection Changes What “Nearby” Means



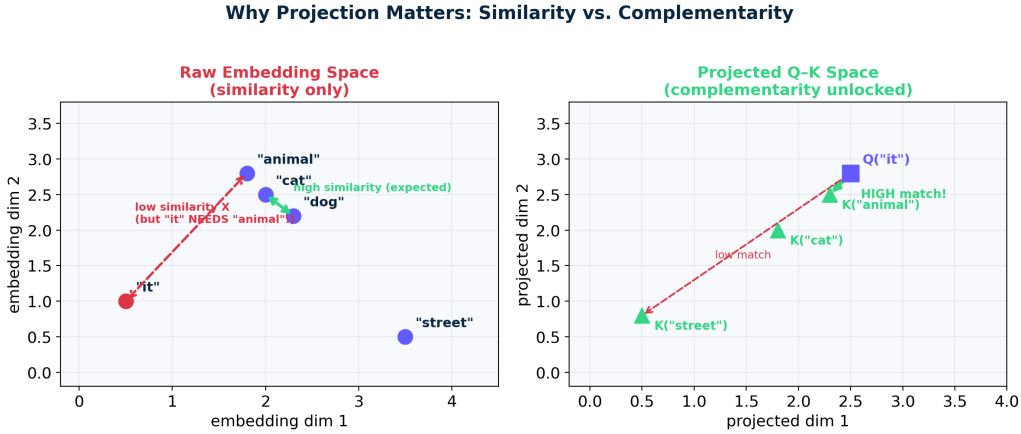
In the embedding space, “it” and “animal” are **far apart**. After projection, they become **neighbours** — the projection reveals that a pronoun *needs* its referent.

Projection as Knowledge Re-Representation



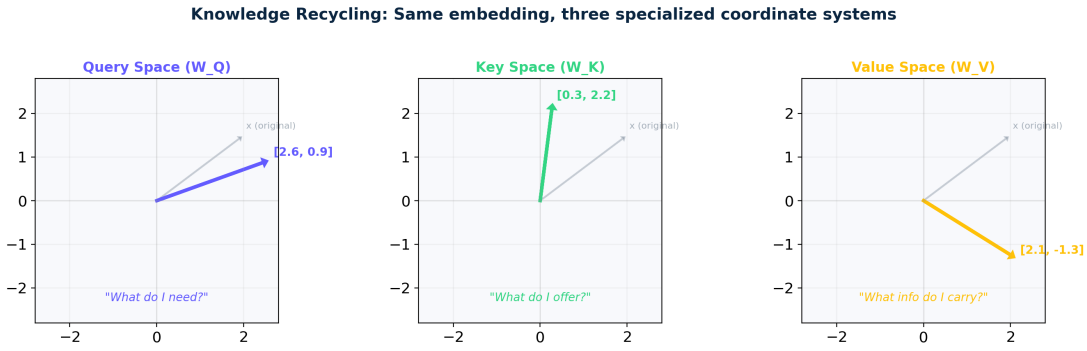
One embedding → three **knowledge representations**. Same knowledge, re-expressed so that **invisible aspects** become dominant.

Why Not Just Use the Raw Embedding?



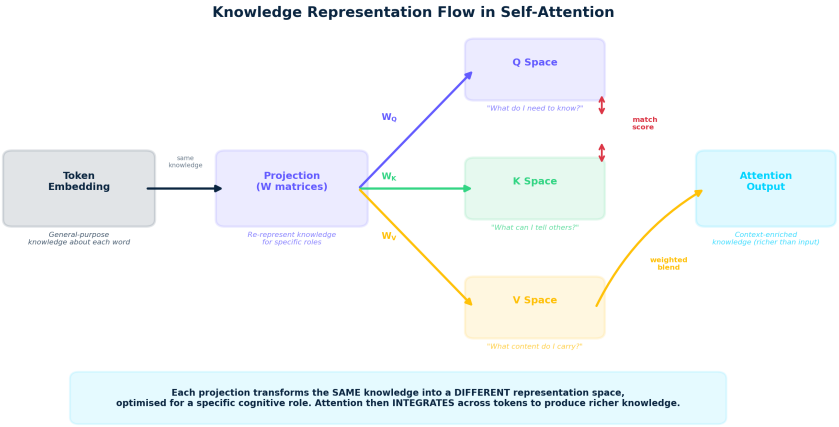
Raw embeddings only match **similar** words. Projections unlock **complementarity**: without re-representation, the model cannot distinguish *what I need* from *what I offer*.

Knowledge Recycling: Same Source, Three Spaces



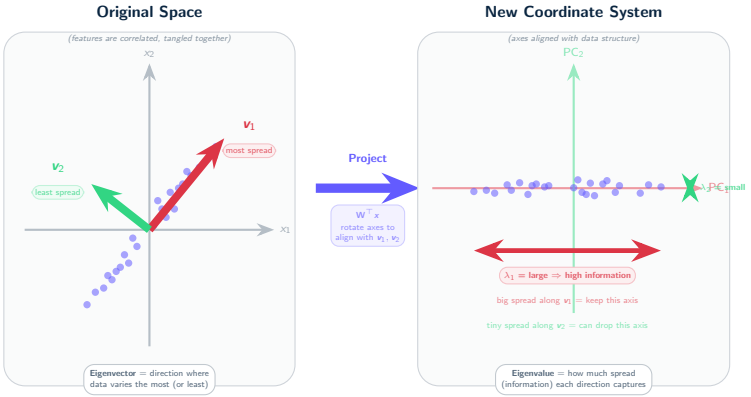
No information is *created* — it is **reorganised** into specialised coordinate systems where different semantic aspects become dominant axes.

The Big Picture: Knowledge Flow in Self-Attention



Embedding → **Projection** (re-represent into Q/K/V) → **Attention** (match & blend) → **Enriched knowledge**.
The output knows more than any single input.

Projection as Knowledge Re-Representation: The PCA Analogy

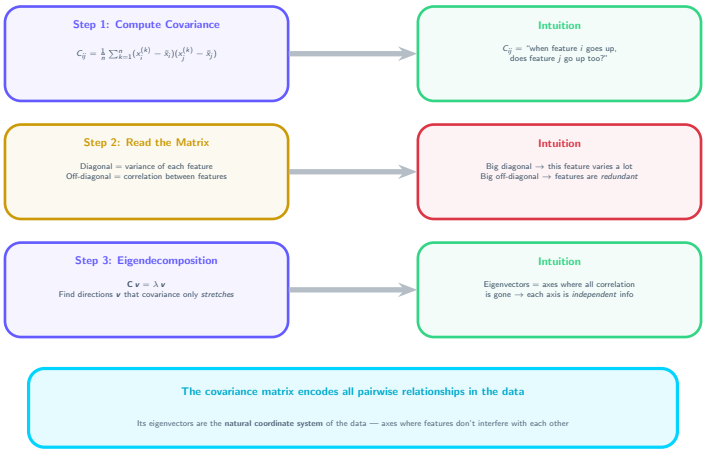


Eigenvectors point where data has the most structure. **Eigenvalues** measure how much. Projecting onto them = **rotating to a better coordinate system**, not losing information.

The Covariance Matrix: Where the Information Lives

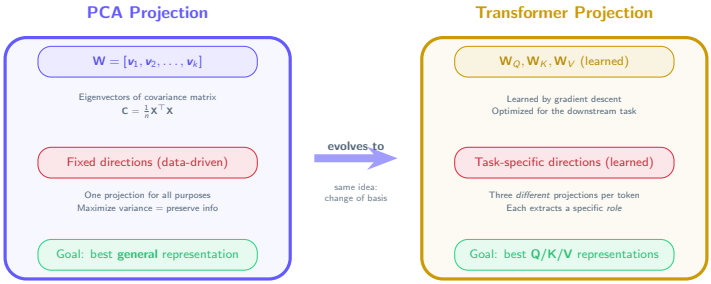
How does PCA know which directions matter?

Answer: it reads the **covariance matrix**



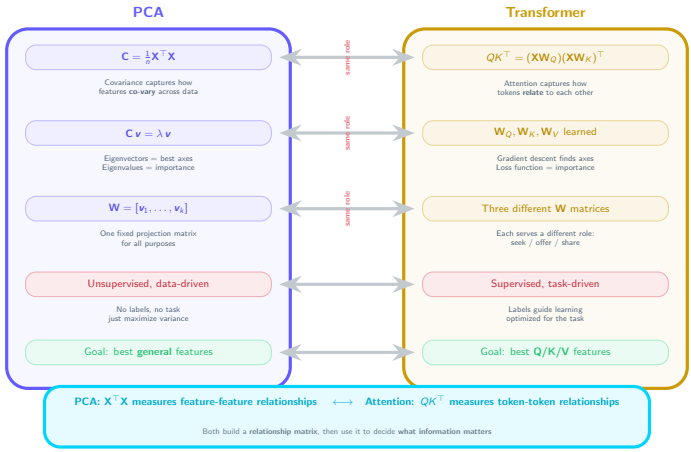
Covariance → **eigenvectors** → **projection**: the matrix finds the relationships, the eigenvectors untangle them, the projection moves you there.

OLD: From PCA to Transformers: Fixed vs Learned Projection



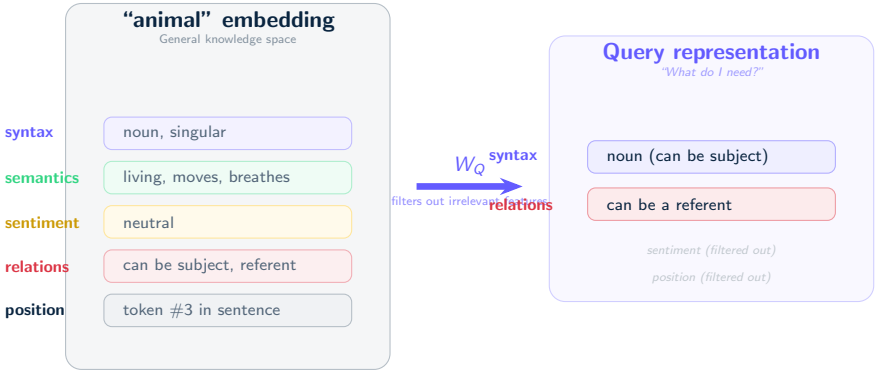
Key insight: PCA finds the *best fixed axes*; transformers learn the *best task-specific axes*. Both are projections — both re-represent knowledge for a purpose.

From Covariance to Attention: The Same Idea



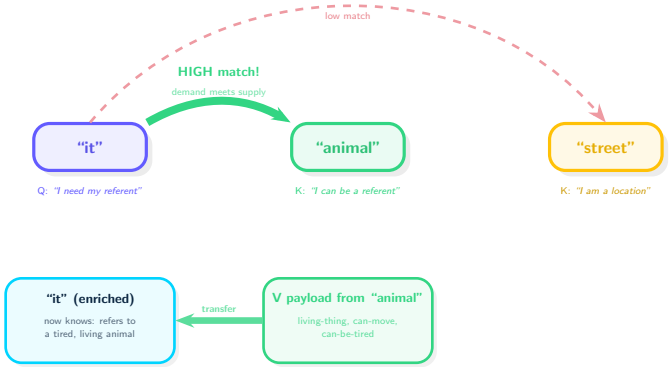
Core parallel: covariance finds fixed axes from data statistics; attention learns dynamic axes from task supervision. Both are **relationship-driven projections**.

Why Projection? Role 1: Feature Extraction



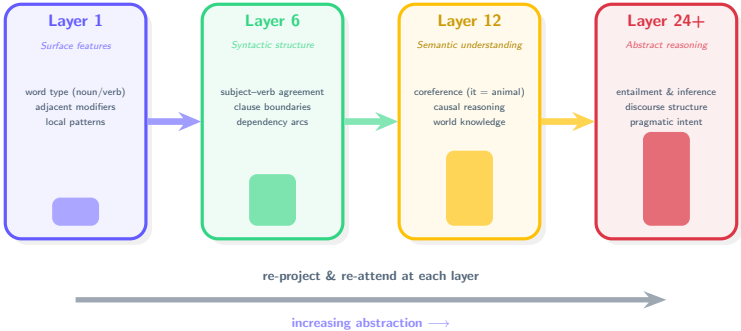
Each **W** is a **learned filter**: the embedding knows *everything*, the projection knows *exactly what it needs*.

Why Projection? Role 2: Contextual Knowledge Enrichment



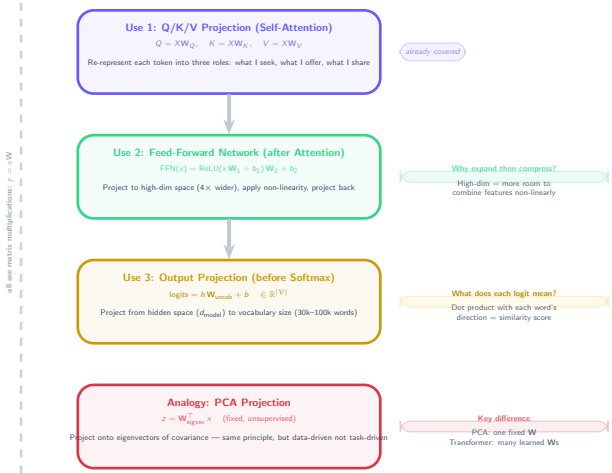
Q/K projections enable **demand–supply matching**. The Value payload transfers knowledge — each token absorbs context it could never access alone.

Why Projection? Role 3: Deeper Language Understanding



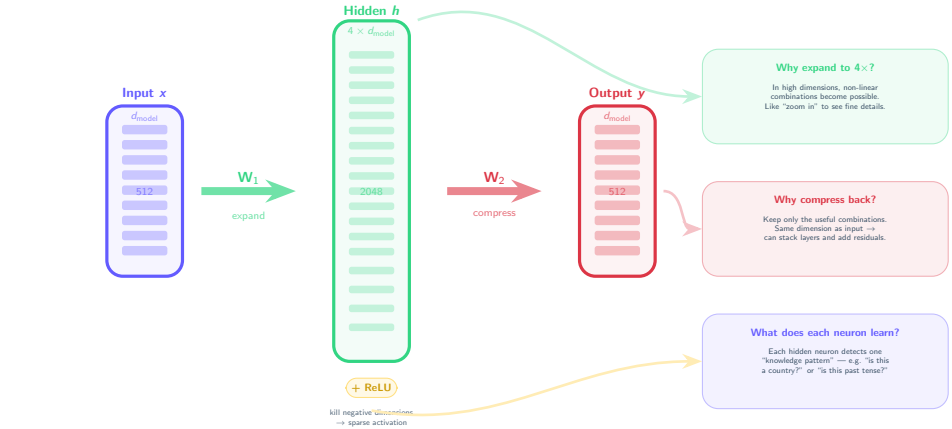
Each layer **re-projects and re-attends**: early layers capture syntax, deeper layers capture semantics and reasoning.

Projection Is Everywhere in the Transformer



Every projection in the transformer is the same operation: multiply by a learned matrix. Q/K/V re-represent for attention; FFN re-represents for reasoning; the final layer re-represents for prediction.

The FFN as Projection: Expand, Transform, Compress



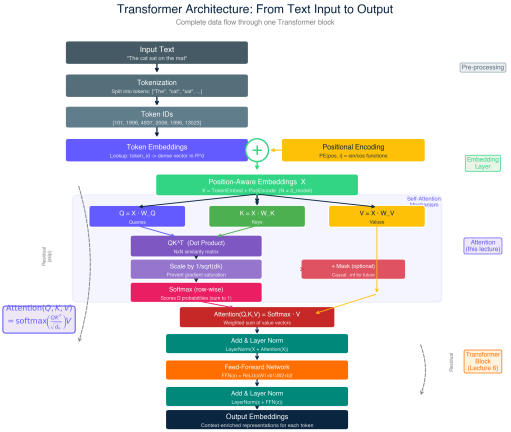
$$\text{FFN}(\boldsymbol{x}) = \text{ReLU}(\boldsymbol{x}\,\mathbf{W}_1 + \boldsymbol{b}_1)\,\mathbf{W}_2 + \boldsymbol{b}_2$$

The FFN is **two projections back-to-back**: $\mathbf{W}_1$ projects *up* to a space where non-linear feature combinations are possible, then $\mathbf{W}_2$ projects *down* keeping only what matters. Same principle as Q/K/V — just a different purpose.

Outline

1. Bridge from Lecture 4: Why Transformers?
2. Positional Encoding
3. Projection as Knowledge Re-Representation
4. Self-Attention: The Core Mechanism
5. Masked Self-Attention
6. Complexity of Self-Attention
7. Summary and Preview

The Transformer: Full Architecture Overview

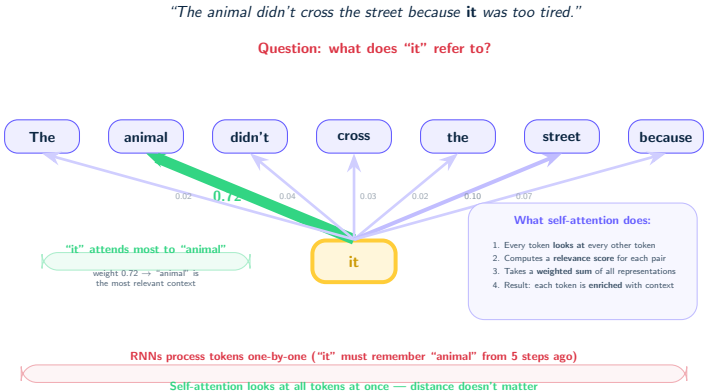


What to notice:

1. **Positional Encoding** is added at the very bottom — the first thing we did today
2. **Self-Attention** is the core of each layer — the main topic of this lecture
3. The encoder (left) uses **bidirectional** attention; the decoder (right) uses **masked** attention
4. Everything else (Add&Norm, FFN, Multi-Head) comes in **Lecture 6**

Today's scope: we covered the two shaded components — PE and single-head self-attention. Next lecture assembles the full block.

Intuition: Why Self-Attention?



Self-attention = each token computes a **weighted sum** of all other tokens, where the weights reflect **relevance**. No sequential bottleneck — every token sees everything.

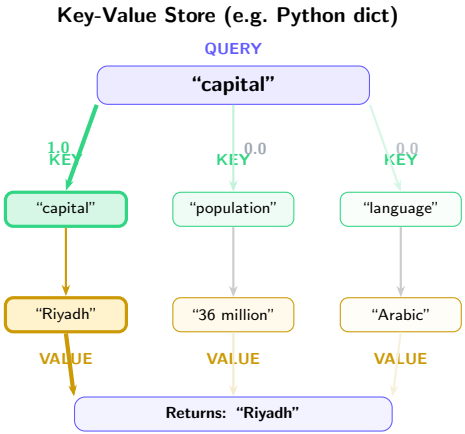
Query, Key, Value: Where Does This Come From?

From Information Retrieval, Not Deep Learning

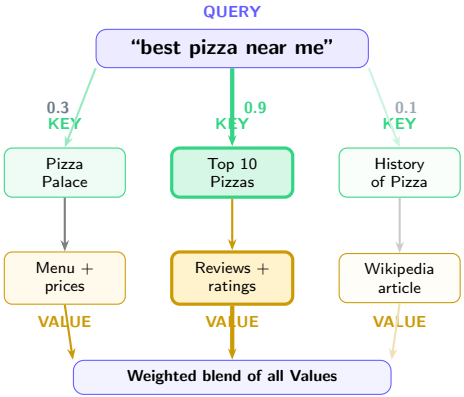
The Q/K/V pattern was **not invented** for Transformers. It comes from **databases and search engines** — decades before deep learning.

Role	What it does
Query	What you are **searching for**
Key	A **label** attached to each item — does it match the query?
Value	The **actual content** you get back when a key matches

In SQL: `SELECT content FROM docs WHERE title MATCHES 'attention'`



Q/K/V in Everyday Life: The Google Search Analogy



How Google Search Uses Q/K/V

1. You type a **Query**: "best pizza near me"
2. Google compares it against millions of **Keys** (page titles)
3. Each key gets a **relevance score** (how well does it match?)
4. The **Values** (page content) of top matches are returned

Result = weighted blend:

$0.9 \times$ "Top 10" $+ 0.3 \times$ "Palace" $+ 0.1 \times$ "History"

Top results get most weight — just like attention!

Key $\neq$ Value: The key is what you match *against* (book title); the value is what you *retrieve* (book content). They are different things.

From Search Engines to Self-Attention: Reusing Q/K/V

In a Search Engine

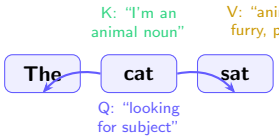
Q What *you* typed
K Page titles (one per page)
V Page content (one per page)

One query, many keys. You search; pages don't search back.

In Self-Attention

Q "What am I looking for?"
K "What do I contain?"
V "What info do I carry?"

Every token is all three at once.
Each word is both the searcher and the result.

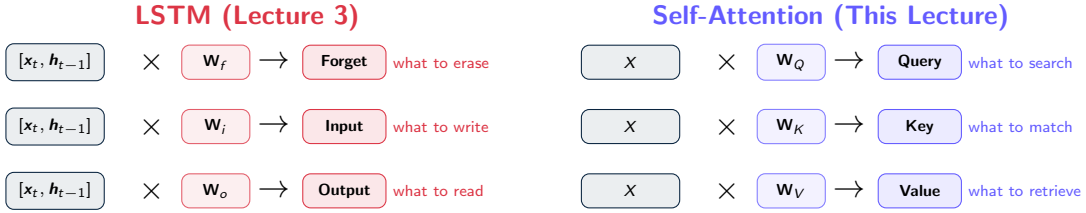


"cat" sends a Query to every token and receives their Values back

The key insight:
In a search engine, Q and K are *different things*. In **self**-attention, the *same* input produces all three — that's why it's called "**self**"-attention.

Recommended video:
3Blue1Brown: "Attention in Transformers"

A Pattern You Already Know: LSTM Gates vs. Q/K/V



Same design pattern: one input × multiple learned projections → multiple specialised roles.
LSTM uses it for *memory control* (forget/write/read). Self-attention uses it for *information retrieval* (search/match/retrieve).

Computing Q, K, V from Input Embeddings

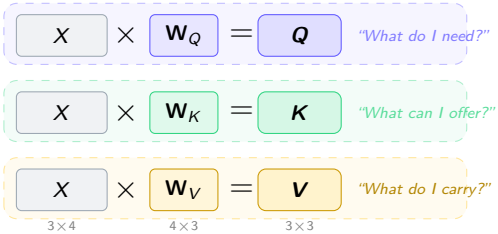
Linear Projections

Each Q, K, V is obtained by multiplying the input embedding X by a **learned weight matrix**:

$$\boldsymbol{Q} = X \cdot \mathbf{W}_Q$$
$$\boldsymbol{K} = X \cdot \mathbf{W}_K$$
$$\boldsymbol{V} = X \cdot \mathbf{W}_V$$

where $\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $\mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$.

Why three views? $\mathbf{Q}$ = "What am I looking for?" $\mathbf{K}$ = "What can I offer?" $\mathbf{V}$ = "What info do I carry?"
Separate projections let the model learn different aspects for matching (Q, K) vs. content (V).



Same knowledge, three **different spaces**
Each $\mathbf{W}$ re-represents X into a **specialised coordinate system** where a different aspect of knowledge becomes dominant.

PyTorch Lab: Computing Q, K, V Projections

Three `nn.Linear` layers project X into Query, Key, and Value — same input, three different views.

```python
import torch
import torch.nn as nn

torch.manual_seed(42)
d_model, d_k, d_v = 4, 3, 3 # same dims as lecture
seq_len = 3 # "I", "like", "AI"

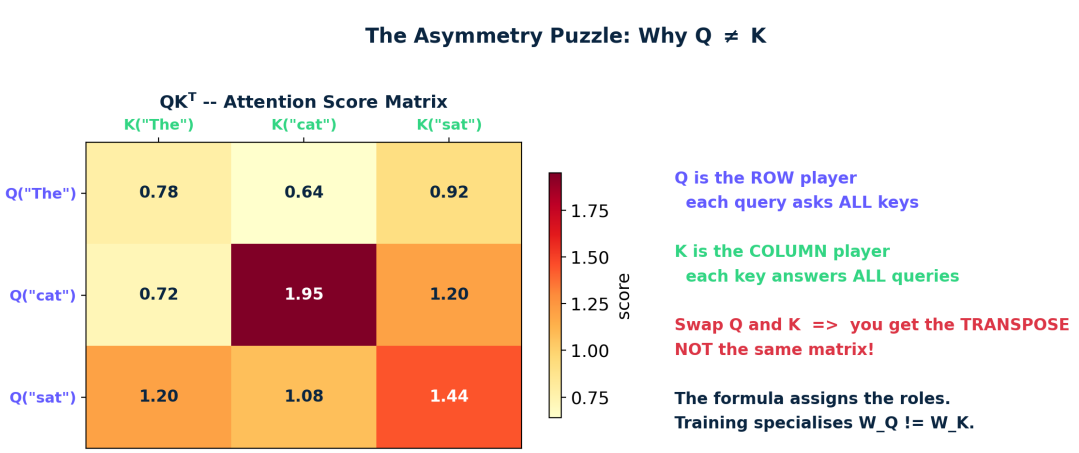
# Three separate learned projections (no bias, matching W_Q * X)
W_Q = nn.Linear(d_model, d_k, bias=False)
W_K = nn.Linear(d_model, d_k, bias=False)
W_V = nn.Linear(d_model, d_v, bias=False)

# X_pos from previous slide: token embeddings + positional encoding
Q = W_Q(X_pos) # (3, 4) x (4, 3) -> (3, 3)
K = W_K(X_pos) # (3, 4) x (4, 3) -> (3, 3)
V = W_V(X_pos) # (3, 4) x (4, 3) -> (3, 3)

print("Q (queries):\n", Q.data.round(decimals=2))
print("K (keys):\n", K.data.round(decimals=2))
print("V (values):\n", V.data.round(decimals=2))
```

`nn.Linear(d_model, d_k, bias=False)` is exactly $\mathbf{W}_Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$. Shape: $(3 \times 4) \rightarrow (3 \times 3)$ — each token goes from 4D to 3D.

The Asymmetry Puzzle: Why Q and K Have Different Names



$\mathbf{W}_Q$ learns "**what I seek**", $\mathbf{W}_K$ learns "**what I provide**". QK^T creates an asymmetric **knowledge marketplace**: seekers (rows) match providers (columns).

Why $QK^\top \neq KQ^\top$: A Numerical Proof

Setup. Two tokens ("cat", "sat"), $d_{\text{model}} = 3$, $d_k = 2$. $\mathbf{W}_Q \neq \mathbf{W}_K$ (different learned weights).

Step 1. Compute $Q = X\mathbf{W}_Q$ and $K = X\mathbf{W}_K$:

$$Q = \underbrace{\begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \end{bmatrix}}_{X} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 3 & 2 \\ 1 & 2 \end{bmatrix} \qquad K = \underbrace{\begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \end{bmatrix}}_{X} \begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 2 & 0 \end{bmatrix}$$

Step 2. Compute both products:

$$QK^\top = \begin{bmatrix} 3 & 2 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 2 & 2 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 8 & \mathbf{6} \\ 4 & 2 \end{bmatrix} \qquad KQ^\top = \begin{bmatrix} 2 & 1 \\ 2 & 0 \end{bmatrix} \begin{bmatrix} 3 & 1 \\ 2 & 2 \end{bmatrix} = \begin{bmatrix} 8 & \mathbf{4} \\ 6 & 2 \end{bmatrix}$$

row i = what token i **seeks** col j = what token j **offers** row i = what token i **offers** col j = what token j **seeks**

Step 3. Compare entry $(1, 2)$ — **different numbers, different meaning**:

$$\underbrace{(QK^\top)_{1,2} = \boldsymbol{q}_{\text{cat}} \cdot \boldsymbol{k}_{\text{sat}} = 3 \cdot 2 + 2 \cdot 0 = \mathbf{6}}_{\text{"cat" seeks} \leftrightarrow \text{"sat" offers}} \quad \neq \quad \underbrace{(KQ^\top)_{1,2} = \boldsymbol{k}_{\text{cat}} \cdot \boldsymbol{q}_{\text{sat}} = 2 \cdot 1 + 1 \cdot 2 = \mathbf{4}}_{\text{"cat" offers} \leftrightarrow \text{"sat" seeks}}$$

$\mathbf{6} \neq \mathbf{4}$: "how much cat seeks sat" $\neq$ "how much cat offers sat". $QK^\top \neq (QK^\top)^\top$ because $\mathbf{W}_Q \neq \mathbf{W}_K \Rightarrow$ **seeking** $\neq$ **offering**.

Reading $QK^\top$: Rows Ask, Columns Answer

$$QK^\top = Q \cdot K^\top = \begin{bmatrix} 3 & 2 \\ 1 & 2 \end{bmatrix} \cdot \begin{bmatrix} 2 & 2 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 8 & 6 \\ 4 & 2 \end{bmatrix}$$

Left matrix = Q (the questioner)

$$Q = \begin{bmatrix} \boldsymbol{q}_{\text{cat}} \\ \boldsymbol{q}_{\text{sat}} \end{bmatrix} = \begin{bmatrix} 3 & 2 \\ 1 & 2 \end{bmatrix}$$

Each row is one token's query

Row 1 ($\boldsymbol{q}_{\text{cat}} = [3, 2]$): "what does **cat** need?"
Row 2 ($\boldsymbol{q}_{\text{sat}} = [1, 2]$): "what does **sat** need?"

The left matrix **drives the attention.**

Right matrix = $K^\top$ (the responder)

$$K^\top = \begin{bmatrix} \boldsymbol{k}_{\text{cat}}^\top & \boldsymbol{k}_{\text{sat}}^\top \end{bmatrix} = \begin{bmatrix} 2 & 2 \\ 1 & 0 \end{bmatrix}$$

Each column is one token's key

Col 1 ($\boldsymbol{k}_{\text{cat}} = [2, 1]$): "what does **cat** offer?"
Col 2 ($\boldsymbol{k}_{\text{sat}} = [2, 0]$): "what does **sat** offer?"

The right matrix **provides the answers.**

rows

columns

Result: $QK^\top$ = Attention Score Matrix

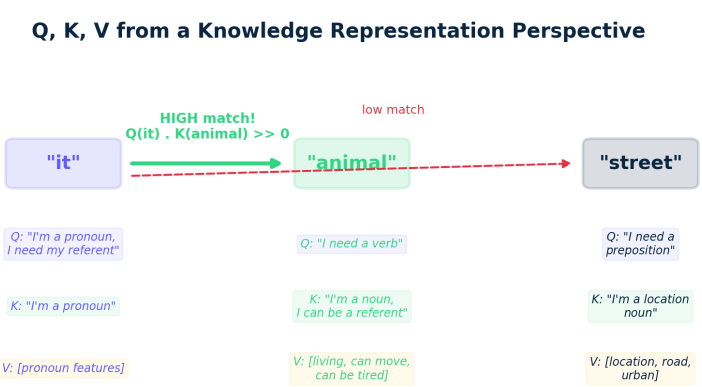
$$QK^\top = \begin{bmatrix} 8 & 6 \\ 4 & 2 \end{bmatrix}$$

$(1,1) = \boldsymbol{q}_{\text{cat}} \cdot \boldsymbol{k}_{\text{cat}} = 8$
cat asks ↔ cat answers

$(1,2) = \boldsymbol{q}_{\text{cat}} \cdot \boldsymbol{k}_{\text{sat}} = 6$
cat asks ↔ sat answers

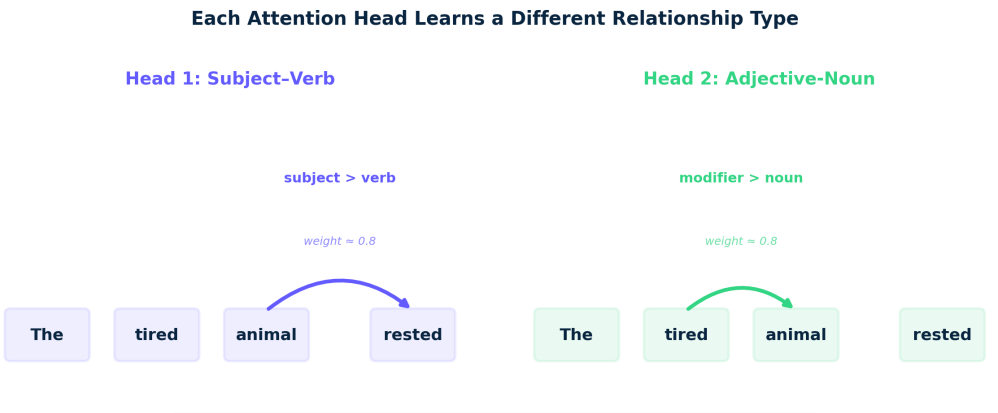
$(2,1) = \boldsymbol{q}_{\text{sat}} \cdot \boldsymbol{k}_{\text{cat}} = 4$
sat asks ↔ cat answers

Q, K, V: Three Knowledge Representations of One Token



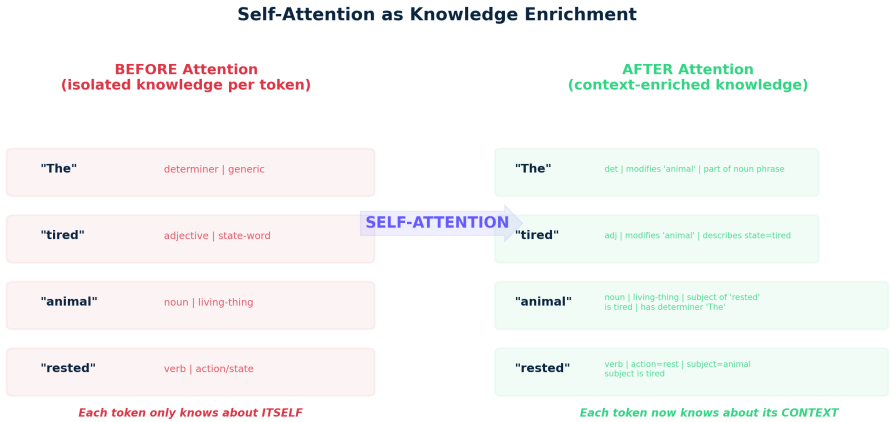
Q = demand (what I lack) **K** = supply (what I offer) **V** = payload (content to transfer). $Q \cdot K =$ **knowledge compatibility score**.

Self-Attention as Knowledge Discovery



Each head discovers a **different knowledge relationship**. Q/K projections are **learned relationship extractors**; attention weights *are* the discovered relationships.

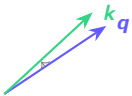
Self-Attention as Knowledge Enrichment



Before: isolated knowledge per token. **After:** contextual knowledge from the entire sentence. The output is strictly **richer** than any single input.

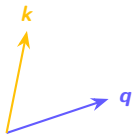
Dot Product as a Similarity Measure

$$\boldsymbol{q} \cdot \boldsymbol{k} = \sum_{i=1}^{d_k} q_i \, k_i = \|\boldsymbol{q}\| \, \|\boldsymbol{k}\| \cos\theta \qquad \text{where } \boldsymbol{q}, \boldsymbol{k} \in \mathbb{R}^{d_k}$$



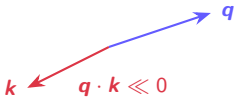
$\boldsymbol{q} \cdot \boldsymbol{k} \gg 0$

High relevance



$\boldsymbol{q} \cdot \boldsymbol{k} \approx 0$

Low relevance



$\boldsymbol{q} \cdot \boldsymbol{k} \ll 0$

Opposite

In self-attention, $\boldsymbol{QK}^T$ computes the dot product between **every query and every key**, producing an $N \times N$ **similarity matrix** that tells the model which tokens are relevant to which.

These diagrams show 2D, but real models use $d_k = 64+$. Same principle: aligned vectors = high dot product, orthogonal = zero. The geometry works identically in any number of dimensions.

The Attention Formula

$$\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{d_k}}\right)\boldsymbol{V}$$

Step 1

Compute $\boldsymbol{Q}\boldsymbol{K}^T$

Dot products between all query–key pairs.
Higher = more relevant.

$$\underbrace{N \times d_k}_{Q} \cdot \underbrace{d_k \times N}_{K^T} = \underbrace{N \times N}_{\text{scores}}$$

Step 2

Scale by $\sqrt{d_k}$

Prevent large dot products from pushing softmax into saturation.

Step 3

Apply softmax

Row-wise: convert scores to probabilities.

$$\text{softmax}(\boldsymbol{z})_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Step 4

Multiply by $\boldsymbol{V}$

Weighted sum of values = context-enriched output.

$$\underbrace{N \times N}_{\text{weights}} \cdot \underbrace{N \times d_v}_{V} = \underbrace{N \times d_v}_{\text{output}}$$

Why Scale by $\sqrt{d_k}$?

The Problem: Dot Products Get Too Big

Imagine 3 students scored on an exam:

	Raw scores	Softmax	Interpretation
Unscaled	48, 2, 1	**1.00**, 0.00, 0.00	"100% sure it's student A"
Scaled	6, 0.25, 0.13	**0.73**, 0.15, 0.12	"Probably A, but consider B & C"

Without scaling: scores like 48 make softmax output ≈ 1.0 for one token and ≈ 0.0 for all others. The model becomes **overconfident** — gradients vanish, it stops learning.

With scaling: dividing by $\sqrt{d_k}$ shrinks the scores. Softmax stays **uncertain** — gradients flow, learning continues.

Why Do Scores Get So Large?

A dot product sums d_k terms:

$$\boldsymbol{q} \cdot \boldsymbol{k} = q_1 k_1 + q_2 k_2 + \cdots + q_{d_k} k_{d_k}$$

More dimensions $\rightarrow$ bigger sum.
If $d_k = 64$: expected magnitude ≈ 8.
If $d_k = 512$: expected magnitude ≈ 23.

The Fix: Divide by $\sqrt{d_k}$

$$\frac{\boldsymbol{q} \cdot \boldsymbol{k}}{\sqrt{d_k}}$$ brings scores back to ≈ 1

Example: $d_k = 64$, $\sqrt{64} = 8$

$$\frac{48}{8} = 6 \qquad \frac{2}{8} = 0.25 \qquad \frac{1}{8} = 0.13$$

Now softmax sees [6, 0.25, 0.13] instead of [48, 2, 1] — **much healthier**.

PyTorch Lab: Attention Scores and Scaling

Steps 1–2 of the attention formula: compute QK^T, then scale by $\sqrt{d_k}$.

```
import torch, math

d_k = 3 # projection dimension

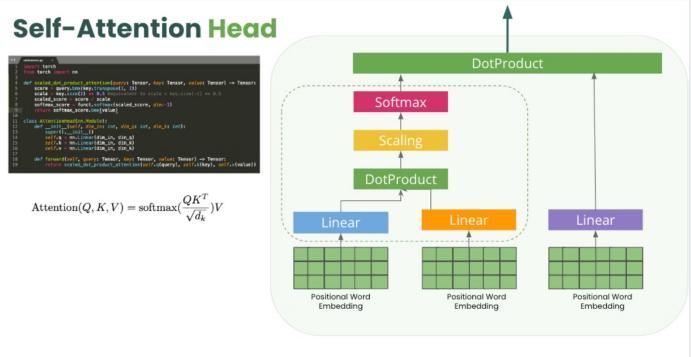
# Step 1: raw attention scores -- dot product of all Q-K pairs
scores = Q @ K.transpose(-2, -1) # (3,3) x (3,3)^T -> (3,3)
print("Raw scores QK^T (3x3 -- one score per token pair):\n",
      scores.data.round(decimals=2))

# Step 2: scale to keep variance ~ 1
scores_scaled = scores / math.sqrt(d_k)
print("\nScaled scores (/ sqrt(3) = / 1.73):\n",
      scores_scaled.data.round(decimals=2))

# Without scaling: large d_k -> large dot products -> softmax saturates
# With scaling: Var(q.k / sqrt(d_k)) = 1 -> healthy gradients
```

`Q @ K.transpose(-2,-1)` computes all N^2 pairwise dot products in one operation. Scaling by $\sqrt{d_k}$ prevents softmax saturation.

Self-Attention Head: Complete Architecture

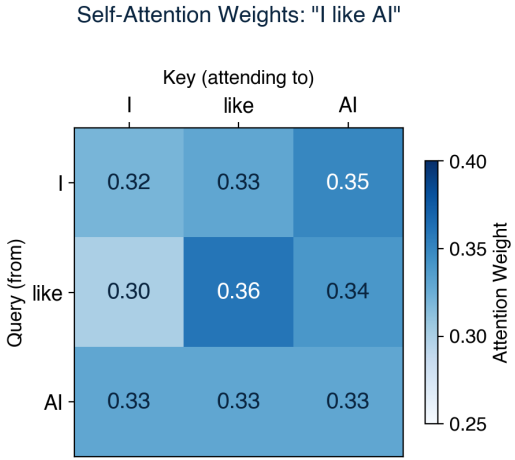


Data flow (bottom to top):

1. **Input** X enters (embeddings + PE)
2. Three projections: $X \to Q, K, V$
3. QK^T: score every token pair
4. Scale by $\sqrt{d_k}$, apply mask (if decoder)
5. Softmax $\to$ attention weights
6. Weights $\times V \to$ context-enriched output

This is **one** attention head. Lecture 6: we run **multiple heads in parallel**, each learning different patterns.

Self-Attention: Computation Overview



How to Read This Matrix

- Each **row** = one token asking "who should I attend to?"
- Each **column** = how much attention that token receives
- Each row **sums to 1** (softmax). **Darker** = more attention

What This Tells Us

- **"I"** → "AI" (0.35): subject looks toward the object
- **"like"** → itself (0.36) + "AI" (0.34): verb finds its object
- **"AI"** → equal (0.33 each): no strong preference

Weights are nearly uniform because $d_k = 3$ is tiny. Real models ($d_k = 64+$) produce much sharper patterns.

Worked Example: Self-Attention on "I like AI" (1/4)

Setup: $d_{\text{model}} = 4$, $d_k = d_v = 3$, sentence = "I like AI"

Step 1: Input Embeddings

$$\underbrace{X}_{3\times 4} = \begin{bmatrix} 0.2 & -0.1 & 0.4 & 0.1 \\ 0.8 & 0.6 & 0.3 & 0.5 \\ 0.5 & 0.4 & 0.2 & 0.7 \end{bmatrix} \quad \leftarrow \text{rows} = \text{I, like, AI}$$

Step 2: Project to Q, K, V $\quad (Q = X\mathbf{W}_Q,\ K = X\mathbf{W}_K,\ V = X\mathbf{W}_V)$

$$\underbrace{Q}_{3\times 3} = \begin{bmatrix} 0.1 & 0.3 & 0.2 \\ 0.5 & 0.4 & 0.1 \\ 0.3 & 0.2 & 0.5 \end{bmatrix} \qquad \underbrace{K}_{3\times 3} = \begin{bmatrix} 0.2 & 0.1 & 0.4 \\ 0.4 & 0.3 & 0.2 \\ 0.1 & 0.5 & 0.3 \end{bmatrix}$$

Each row is a 3-dimensional vector for one token.
Q encodes "what each token is looking for"; K encodes "what each token offers."

Worked Example: Self-Attention on "I like AI" (2/4)

Step 3: Compute Attention Scores QK^T

Row 1 ("I") × Col 1 ("I"): $0.1(0.2) + 0.3(0.1) + 0.2(0.4) = \mathbf{0.13}$
Row 2 ("like") × Col 2 ("like"): $0.5(0.4) + 0.4(0.3) + 0.1(0.2) = \mathbf{0.34}$

$$\underbrace{QK^T}_{3\times 3} = \begin{bmatrix} 0.13 & 0.17 & 0.22 \\ 0.18 & \mathbf{0.34} & 0.28 \\ 0.28 & 0.28 & 0.28 \end{bmatrix} \quad \text{(one score per token pair)}$$

Step 4: Scale by $\sqrt{d_k} = \sqrt{3} \approx 1.73$

$$\frac{QK^T}{\sqrt{d_k}} = \begin{bmatrix} 0.075 & 0.098 & 0.127 \\ 0.104 & 0.196 & 0.162 \\ 0.162 & 0.162 & 0.162 \end{bmatrix}$$

Scaling keeps values small → softmax won't saturate.

Worked Example: Self-Attention on "I like AI" (3/4)

Step 5: Apply Softmax (row-wise)

$$\text{Attention weights} \approx \begin{bmatrix} 0.32 & 0.33 & \mathbf{0.35} \\ 0.30 & \mathbf{0.36} & 0.34 \\ 0.33 & 0.33 & 0.33 \end{bmatrix}$$

Reading the Attention Matrix

- **Row 1 ("I"):** attends slightly more to "AI" (0.35)
- **Row 2 ("like"):** attends most to itself (0.36) and "AI" (0.34) — capturing verb–object relation
- **Row 3 ("AI"):** attends uniformly (0.33 each)

	I	like	AI
I	.32	.33	.35
like	.30	**.36**	.34
AI	.33	.33	.33

Darker = higher attention weight

Why are the weights nearly uniform? Our toy example has only 3 tokens and $d_k = 3$ — the scaled scores are all close to zero, so softmax spreads weight evenly. In real models ($d = 768$, long sentences), attention becomes **much sharper**: some tokens get 0.6+ weight while others drop to near 0.

Worked Example: Self-Attention on "I like AI" (4/4)

Step 6: Multiply by V to get context-enriched output

Suppose: $\underbrace{V}_{3\times 3} = \begin{bmatrix} 0.3 & 0.1 & 0.2 \\ 0.6 & 0.4 & 0.3 \\ 0.4 & 0.5 & 0.6 \end{bmatrix}$

Output for "like" (row 2): $0.30 \cdot V_{\text{I}} + 0.36 \cdot V_{\text{like}} + 0.34 \cdot V_{\text{AI}}$

$$= 0.30 \begin{bmatrix} 0.3 \\ 0.1 \\ 0.2 \end{bmatrix} + 0.36 \begin{bmatrix} 0.6 \\ 0.4 \\ 0.3 \end{bmatrix} + 0.34 \begin{bmatrix} 0.4 \\ 0.5 \\ 0.6 \end{bmatrix} = \underbrace{\begin{bmatrix} 0.442 \\ 0.344 \\ 0.372 \end{bmatrix}}_{\in \mathbb{R}^{d_v}}$$

The output for "like" is now a **context-enriched vector** — a blend of all tokens, weighted by relevance. It "knows" its object is "AI."

PyTorch Lab: Softmax and Weighted Output

Steps 3–4 as a reusable function:

```python
import torch.nn.functional as F

def apply_attention(scores_scaled, V):
    # Step 3: softmax (row-wise)
    weights = F.softmax(scores_scaled,
                        dim=-1)
    # Step 4: weighted sum of values
    output = weights @ V
    return output, weights

# --- Call the function ---
out, attn = apply_attention(
    scores_scaled, V)
print("Weights:\n",
      attn.data.round(decimals=2))
print("Output:\n",
      out.data.round(decimals=2))
```

Expected Output: Attention Weights

	I	like	AI
I	0.32	0.33	0.35
like	0.30	0.36	0.34
AI	0.33	0.33	0.33

Expected Output: Attention Output

```
I     [0.434, 0.340, 0.368]
like  [0.442, 0.344, 0.372]
AI    [0.433, 0.333, 0.367]
```

Interpretation

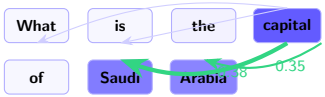
- **"like"** output $\approx 0.36 \times V_{\text{like}} + 0.34 \times V_{\text{AI}} + 0.30 \times V_{\text{I}}$
  $\rightarrow$ mostly itself + its object "AI"
- **"I"** leans slightly toward "AI" (0.35)
  $\rightarrow$ subject finds the object
- **"AI"** spreads evenly (0.33 each)
  $\rightarrow$ no dominant relationship in 3 tokens

The "Capital of Saudi Arabia" Example

Sentence: "What is the **capital** of Saudi Arabia?"

Tracing attention for the query word "capital":

1. Compute Q for "capital" — encodes "looking for a place"
2. Compute K for all words
3. Dot product $Q \cdot K$:
   - ✓ "Saudi" → **high score**
   - ✓ "Arabia" → **high score**
   - ✗ "What", "is" → low scores
4. After softmax: "Saudi" and "Arabia" get highest weights
5. Output: "capital" is enriched with **Saudi Arabia** context



"capital" attends most to "Saudi" and "Arabia"

Attention Heatmap: "Capital of Saudi Arabia"



Key observations:

- **"capital"** attends strongly to **"Saudi"** and **"Arabia"** — it learned *which country* it refers to
- **"Saudi"** and **"Arabia"** attend to each other — the model binds them as one entity
- **"the"** attends to **"capital"** — the article finds its noun

Self-attention discovers **semantic relationships** (capital ↔ country) without any explicit grammar rules.

Self-Check Q2

In the worked example, the attention weights were relatively uniform (≈ 0.33). What would the weights look like if "like" were highly relevant **only to "AI"**? How would the output change?

Think about what "sharp" vs. "flat" attention distributions mean for the output vector...

Outline

1. Bridge from Lecture 4: Why Transformers?
2. Positional Encoding
3. Projection as Knowledge Re-Representation
4. Self-Attention: The Core Mechanism
5. Masked Self-Attention
6. Complexity of Self-Attention
7. Summary and Preview

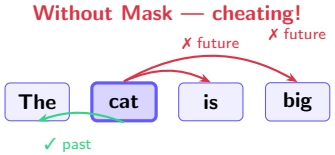
Why Masking? The Autoregressive Constraint

The Problem

In **generative** models (GPT), the model predicts the next token from **previous tokens only**. During training, we process all tokens at once — but each token must **not see the future**.

Without mask: token 2 sees tokens 3, 4...
⇒ **Cheating!** Memorises, not learns.

With causal mask: token 2 sees only 1, 2.
Future tokens are **invisible**.



The Causal Mask: How It Works

The Mask Matrix

For 4 tokens, the causal mask is:

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Applied: $\text{scores} = \frac{QK^T}{\sqrt{d_k}} + M$

After softmax:

- Token 1 $\rightarrow$ only itself
- Token 2 $\rightarrow$ tokens 1–2
- Token 3 $\rightarrow$ tokens 1–3
- Token 4 $\rightarrow$ tokens 1–4

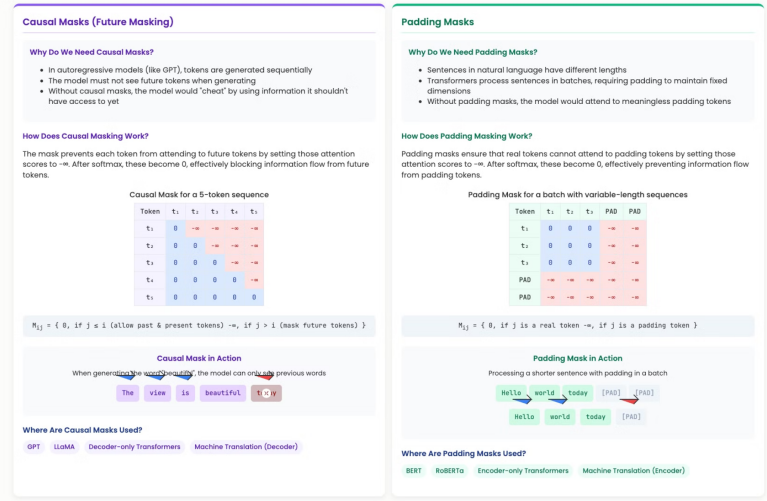
Why $-\infty$ Works

$\text{softmax}(-\infty) = \frac{e^{-\infty}}{\sum} = 0$

Masked positions get **exactly zero** weight.

	t1	t2	t3	t4
t1	✓	$-\infty$	$-\infty$	$-\infty$
t2	✓	✓	$-\infty$	$-\infty$
t3	✓	✓	✓	$-\infty$
t4	✓	✓	✓	✓

Causal Masks vs. Padding Masks



Types of Attention Masks: Summary

Types of Attention Masks

Causal (Future) Mask
Prevents looking at future tokens

Padding Mask
Ignores padding tokens in batches

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_k} + M)V$$

M is the mask matrix with different values for causal and padding masks

Causal Masks: Preventing "Cheating" in Generation

	t_1	t_2	t_3	t_4
t_1	0	$-\infty$	$-\infty$	$-\infty$
t_2	0	0	$-\infty$	$-\infty$
t_3	0	0	0	$-\infty$
t_4	0	0	0	0

Triangular mask: each position only sees itself and previous positions

Causal Mask in Action



When predicting "is", model can see "The" and "cat" but not "sleeping"

- Used in autoregressive text generation
- Prevents "cheating" by looking ahead

$M_{ij} = \{ 0, \text{if } j \le i \text{ (past \& present)} \; -\infty, \text{if } j > i \text{ (future)} \}$

GPT LLaMA Decoder-only models

Padding Masks: Ignoring Irrelevant Padding

	t_1	t_2	t_3	PAD
t_1	0	0	0	$-\infty$
t_2	0	0	0	$-\infty$
t_3	0	0	0	$-\infty$
PAD	$-\infty$	$-\infty$	$-\infty$	$-\infty$

Padding mask: real tokens attend to real tokens only, not padding

Padding Mask in Action



"Hello" can attend to real words but not to padding tokens

- Used for variable-length sequences in batches
- Prevents attending to meaningless padding

$M_{ij} = \{ 0, \text{if } j \text{ is a real token} \; -\infty, \text{if } j \text{ is a padding token} \}$

BERT RoBERTa Encoder-only models

Masked Self-Attention: Try It Yourself

Sentence: "The cat is sleeping" (4 tokens, predicting autoregressively)

Here are the **unmasked** attention scores (after scaling):

$$S = \frac{QK^T}{\sqrt{d_k}} = \begin{bmatrix} 1.2 & 0.5 & 0.3 & 0.8 \\ 0.7 & 1.5 & 0.9 & 0.4 \\ 0.3 & 0.8 & 1.1 & 0.6 \\ 0.5 & 0.4 & 0.7 & 1.3 \end{bmatrix}$$

Your task:

1. Which positions should be masked with $-\infty$? (Draw the mask pattern)
2. After masking, what happens to row 1 ("The")? How many tokens can it see?
3. What will the softmax weights look like for row 1?

Think for 30 seconds, then we'll work through it together...

Masked Self-Attention: Solution

Step 1: Add causal mask ($-\infty$ for future positions = upper triangle):

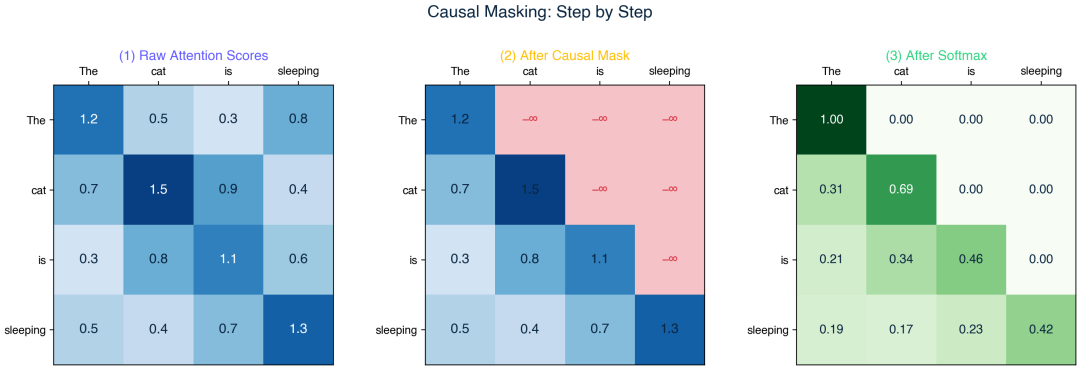
$$S + M = \begin{bmatrix} 1.2 & -\infty & -\infty & -\infty \\ 0.7 & 1.5 & -\infty & -\infty \\ 0.3 & 0.8 & 1.1 & -\infty \\ 0.5 & 0.4 & 0.7 & 1.3 \end{bmatrix}$$

Step 2: Softmax (row-wise, $-\infty \to 0$, rest renormalised):

$$\text{Attention} = \begin{bmatrix} \mathbf{1.0} & 0 & 0 & 0 \\ 0.31 & \mathbf{0.69} & 0 & 0 \\ 0.19 & 0.31 & \mathbf{0.42} & 0 \\ 0.18 & 0.16 & 0.22 & \mathbf{0.40} \end{bmatrix} \quad \leftarrow \text{lower triangle only}$$

"The" sees only itself → weight = 1.0. **"cat"** sees The + itself → 0.31 + 0.69. Each row sums to 1.
Future tokens are **invisible** — exactly what autoregressive generation needs.

Causal Masking: Visual Step-by-Step



(1) Raw scores show all pairwise similarities. **(2)** Future positions masked with $-\infty$. **(3)** After softmax: masked positions $\to 0$, rest renormalised.

PyTorch Lab: Applying a Causal Mask

Build the mask matrix with $-\infty$, apply it before softmax, and see future positions zeroed out.

```python
import torch
import torch.nn.functional as F

seq_len = 3 # "I", "like", "AI"

# Upper triangle = -inf (block future tokens)
mask = torch.triu(
    torch.ones(seq_len, seq_len) * float('-inf'),
    diagonal=1
)
print("Causal mask:\n", mask)

# Apply to scaled scores from earlier
masked_scores = scores_scaled + mask
print("\nMasked scores:\n", masked_scores.data.round(decimals=2))

# Softmax: -inf becomes 0, rest renormalises
causal_weights = F.softmax(masked_scores, dim=-1)
print("\nCausal attention weights:\n",
      causal_weights.data.round(decimals=2))

causal_output = causal_weights @ V # masked output
```

Token 1 ("I") sees only itself (weight = 1.0). Token 2 sees tokens 1–2. Token 3 sees all three. Future positions get **exactly zero** weight.

Self-Check Q3

Why do encoder-only models like BERT **not** use causal masks? What advantage does **bidirectional attention** give them?

Think about what BERT is trained to do (MLM) and what information it needs...

Outline

1. Bridge from Lecture 4: Why Transformers?
2. Positional Encoding
3. Projection as Knowledge Re-Representation
4. Self-Attention: The Core Mechanism
5. Masked Self-Attention
6. Complexity of Self-Attention
7. Summary and Preview

The $O(N^2)$ Problem

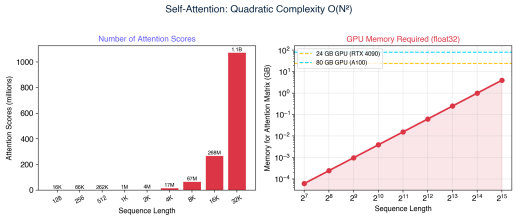
Where Does N^2 Come From?

The matrix multiplication $\boldsymbol{QK}^T$ computes dot products between **every pair of tokens**:

- $\boldsymbol{Q}$: shape $(N \times d_k)$
- $\boldsymbol{K}^T$: shape $(d_k \times N)$
- $\boldsymbol{QK}^T$: shape $(N \times N)$

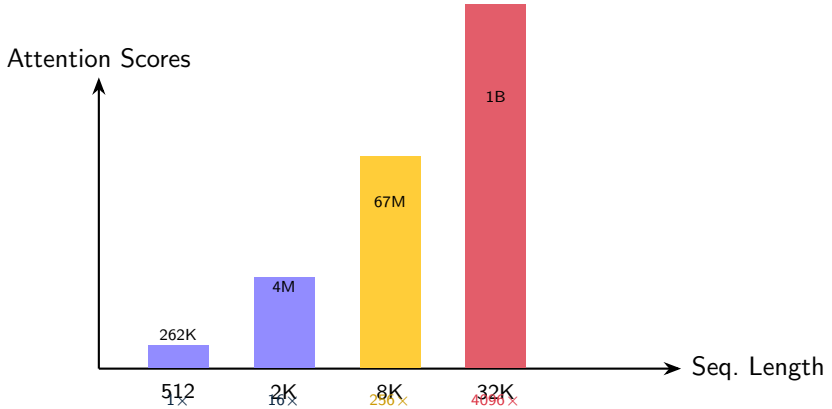
Complexity

Time: $O(N^2 \cdot d_k)$ — quadratic in sequence length
Memory: $O(N^2)$ — must store full attention matrix



Every token attends to every other token.
N tokens $\rightarrow$ N^2 attention scores.

Why Quadratic Complexity Is a Problem



Doubling the sequence length **quadruples** the computation. This is why long-document processing is expensive — and why **efficient attention** (Lecture 6) is critical.

	512	2K	8K	32K
Params	1.4 GB	1.4 GB	1.4 GB	1.4 GB
Attention	**1 MB**	**16 MB**	**256 MB**	**4 GB**

Params = constant. Attention = **quadratic**. A100 = 80 GB.

Self-Check Q4

A model processes a 100,000-token document with standard self-attention.

How many attention scores must be computed?

Why is this impractical even on modern GPUs?

Hint: think about memory requirements in float32...

Self-Check Q5

What would happen if we removed positional encoding **and** the causal mask from a Transformer decoder? How would the model's behaviour change?

Consider: would the model know word order? Could it generate text left-to-right?

PyTorch Lab: Putting It All Together

Complete single-head self-attention in one function — every concept from this lecture.

```python
import torch, torch.nn as nn, torch.nn.functional as F, math
def self_attention(X, W_Q, W_K, W_V, mask=None):
    Q, K, V = W_Q(X), W_K(X), W_V(X) # project to Q, K, V
    d_k = Q.size(-1)
    scores = Q @ K.transpose(-2, -1) / math.sqrt(d_k) # scaled scores
    if mask is not None:
        scores = scores + mask # -inf blocks future
    weights = F.softmax(scores, dim=-1) # attention weights
    return weights @ V, weights # output, weights
# --- Run on "I like AI" ---
torch.manual_seed(42)
d_model, d_k = 4, 3
W_Q = nn.Linear(d_model, d_k, bias=False)
W_K = nn.Linear(d_model, d_k, bias=False)
W_V = nn.Linear(d_model, d_k, bias=False)
X = torch.tensor([[0.2,-0.1,0.4,0.1], # I
                  [0.8, 0.6,0.3,0.5], # like
                  [0.5, 0.4,0.2,0.7]]) # AI
mask = torch.triu(torch.ones(3,3) * float('-inf'), diagonal=1)
out, attn = self_attention(X, W_Q, W_K, W_V, mask)
print("Output:\n", out.data.round(decimals=2))
print("Attention:\n", attn.data.round(decimals=2))
```

Everything from this lecture in one function: Q/K/V projection, scaling, causal mask, softmax, weighted output. In Lecture 6 we wrap **multiple heads** around it.

Outline

1. Bridge from Lecture 4: Why Transformers?
2. Positional Encoding
3. Projection as Knowledge Re-Representation
4. Self-Attention: The Core Mechanism
5. Masked Self-Attention
6. Complexity of Self-Attention
7. Summary and Preview

Key Equations Reference Card

Component	Equation	Purpose
Sinusoidal PE	$\text{PE}(\text{pos}, 2i) = \sin(\text{pos}/10000^{2i/d})$	Fixed position encoding
Q, K, V	$Q = XW_Q,\ K = XW_K,\ V = XW_V$	Learned projections
Attention	$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$	Weighted aggregation
Causal Mask	Add $-\infty$ to future positions	Block future tokens
Complexity	$O(N^2 \cdot d_k)$ time, $O(N^2)$ memory	Quadratic bottleneck

What We Covered Today

1. Positional Encoding

We **inject position** via PE vectors added to embeddings.

- **Sinusoidal:** fixed formula, generalises to any length
- **Learned:** flexible, adapts to the task
- **Trend:** APE $\rightarrow$ RPE $\rightarrow$ RoPE

2. Self-Attention

Every token asks "*who is relevant to me?*" and gets a **weighted blend** of answers.

- Three views: Q (search), K (match), V (content)
- Scaled dot-product prevents softmax saturation

3. Masked Self-Attention

No "cheating" — block future tokens during generation.

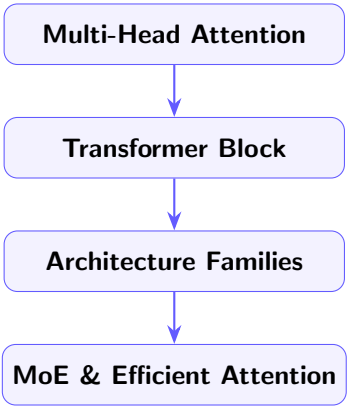
- **Causal mask:** $-\infty$ on future positions
- **Padding mask:** $-\infty$ on <PAD> tokens

4. Complexity

Every token attends to every other $\rightarrow$ $O(N^2)$ cost.

- 32K tokens = 1 billion attention scores
- Efficient attention is essential (next lecture)

Preview: Lecture 6



Today's self-attention is the **building block** for everything in Lecture 6:
component $\rightarrow$ assembly $\rightarrow$ variants $\rightarrow$ scaling.

Questions?

Thank you!

Prof. Anis Koubaa
akoubaa@alfaisal.edu

Alfaisal University

Appendix: Self-Check Solutions (1/3)

Q1: Sinusoidal vs. Learned PE Generalisation

Sinusoidal encoding uses a **deterministic formula** that computes PE(pos) for *any* value of pos. Even position 10,000 works, regardless of training length.
Learned embeddings are a **lookup table** of trainable vectors (one per position). If trained with max length 512, position 513 has no embedding — the model cannot handle it.

Q2: Sharp vs. Uniform Attention

If "like" were highly relevant only to "AI":

- Weights: $\approx [0.1, 0.1, \mathbf{0.8}]$ instead of $[0.30, 0.36, 0.34]$
- Output for "like" $\approx 0.8 \cdot V_{\text{AI}} + 0.1 \cdot V_{\text{I}} + 0.1 \cdot V_{\text{like}}$
- "like" becomes dominated by "AI"'s value vector — highly context-specific
- Uniform weights produce a generic blend; sharp weights produce specialised output

Appendix: Self-Check Solutions (2/3)

Q3: Why No Causal Mask in BERT?

BERT uses **Masked Language Modeling**: it masks random tokens and predicts them from *both* left and right context. A causal mask would block right-context, defeating the purpose.
Example: "the river **[MASK]** eroded" — BERT needs *both* "river" and "eroded" to predict "bank." Bidirectional attention enables richer representations.

Q4: 100K Tokens

$100{,}000 \times 100{,}000 = 10^{10}$ (10 billion) attention scores.
In float32: $10^{10} \times 4$ bytes $= 40$ GB — just for *one* attention matrix in *one* layer. A single GPU typically has 24–80 GB, and this is *before* accounting for model weights, gradients, multiple layers, and multiple heads. Completely impractical without efficient attention methods.

Appendix: Self-Check Solutions (3/3)

Q5: Removing PE and Causal Mask

Without positional encoding:

- Self-attention becomes a **set operation** — treats input as an unordered bag
- "The cat sat on the mat" = "The mat sat on the cat" (identical representations)
- Useless for virtually all language tasks

Without causal mask:

- During training, each token "sees" future tokens — effectively **cheating**
- The model cannot learn to generate left-to-right
- At inference, there *are* no future tokens, so the model collapses

Without *both*: the model has no sense of order *and* no generation capability. It reduces to a context-free, non-directional averaging function — essentially useless.